
HTV IP Control Protocol

Version 2.8

Samsung Electronics
TV SW Lab, VD

Table Of Contents

1. Overview

1.1 Overview

1.2 Device Discovery

1.2.1. Device Type

1.2.2. Device Model

1.2.3. Device Description

1.3. Connection

1.4. Security

1.4.1. Cipher Suites

1.5. Data Format

1.6. Server Error Code for JSON-RPC

2. Reference

3. Control

3.1. TV Control List

3.1.1. Power Control

3.1.2. Volume Control

3.1.3. Mute Control

3.1.4. Channel Control

3.1.5. Source Control

3.1.6. Picture Control

3.1.7. Sound Control

3.1.8. Status

3.1.9. Remote Key Control

3.1.10. App Control

3.1.11. First Screen App Control

3.1.12. Art Mode Control

3.1.13. MultiView Control

3.1.14. Display Rotator Control

3.1.15. Hotel TV Control

4. Revision History

5. Appendix

5.1. TV menu tree change for 2020 TV

5.2. TV menu name change for 2020 TV

1. Overview

1.1. Overview

Samsung TV supports IP control functions using JSON-RPC 2.0 format message exchanging. This specification does not specify the way of discovery for the IP control service on the TV. CI client may have manual input GUI for TV IP address or UPnP device discovery.

1.2. Device Discovery

The IP control supports "device discovery" feature using UPnP technology. UPnP service is optional since this specification provides relevant IP control command.

1.2.1. Device Type

urn:samsung.com:device:IPControlServer:1

1.2.2. Device Model

Device Type	Root	Req. or Opt	ServiceType	Req. or Opt.	Service ID
IPControlServer:1	YES	R	IPControlService:1.0	O	IPControlService

1.2.3. Device Description

```
<?xml version="1.0"?>
<root xmlns="urn:samsung.com:device-1-0" xmlns:sec="http://www.sec.co.kr/dlna" >
  <specVersion>
    <major>1</major>
    <minor>0</minor>
  </specVersion>
  <device>
    <deviceType>urn:samsung.com:device:IPControlServer:1</deviceType>
    <friendlyName>short user-friendly title</friendlyName>
    <manufacturer>Samsung Electronics</manufacturer>
    <manufacturerURL>http://www.samsung.com/sec</manufacturerURL>
    <modelDescription>Samsung TV 2018</modelDescription>
    <modelName>device model name</modelName>
    <modelNameNumber>1.0</modelNameNumber>
    <modelURL>http://www.samsung.com/sec</modelURL>
  </device>
</root>
```

```

    <serialNumber>serialNumber</serialNumber>
    <UDN>uuid:UUID</UDN>
  <iconList>
    <icon>
      <mimetype>image/format</mimetype>
      <width>horizontal pixels</width>
      <height>vertical pixels</height>
      <depth>color depth</depth>
      <url>URL to icon</url>
    </icon>
  </iconList>
  <serviceList>
    <service>
      <serviceType>urn:samsung.com:service:IPControlService:1</serviceType>
      <serviceId>urn:samsung.com:serviceId:IPControlService</serviceId>
      <controlURL>/upnp/control/IPControlService1</controlURL>
      <eventSubURL>/upnp/event/IPControlService1</eventSubURL>
      <SCPDURL>/IPControlService1_1.xml</SCPDURL>
    </service>
  </serviceList>
</device>
</root>

```

1.3. Connection

Samsung TV IP control server listens for IP control messages using fixed port number of below table. The maximum number of clients that can be connected to TV IP control server is 5.

Year	TV	Wall Pro	Wall Lux
~2017	1515	N/A	N/A
2018,2019	1515	1516	1516
2020~	1516	1516	1516

1.4. Security

HTTP over TLS(TLS v.1.2) is required for controlling TV by using IP control messages.

<https://en.wikipedia.org/wiki/HTTPS>

<https://tools.ietf.org/html/rfc5246>

1.4.1. Cipher Suites

Below cipher suites are not available for TV IPControl server due to weak and vulnerable security.

```
TLS_RSA_WITH_RC4_128_MD5
TLS_RSA_WITH_RC4_128_SHA
TLS_RSA_WITH_SEED_CBC_SHA
TLS_DHE_RSA_WITH_3DES_EDE_CBC_SHA
TLS_RSA_WITH_3DES_EDE_CBC_SHA
```

1.5. Data Format

The maximum HTTP body size including header is 200KB.

TV IP control server supports chunked encoding.

JSON-RPC 2.0 format is used for IP control message exchange.

<http://www.jsonrpc.org/specification>

Client uses POST request with following informations in the header:

- **Content-Type** : MUST be **application/json**
- **Content-Length** : MUST contain the correct length according to the HTTP-specification
- **Accept** : MUST be **application/json**

```
POST / HTTP/1.1
Host : 192.168.0.5:<PORT NUMBER>
Content-Type : application/json
Content-Length: ...
Accept: application/json
{
  "jsonrpc" : "2.0",
  "method" : "sum",
  "params" : { "b" : 34, "c" : 56, "a" : 12},
  "id": 123
}
```

Server should respond with following informations in the header:

- **Content-Type**: MUST be **application/json**
- **Content-Length**: MUST contain the **correct length** according to the HTTP specification.

The status codes are:

- **200 OK**
for responses (both for Response and Error objects)
- **204 No Response / 202 Accepted**
for empty responses, i.e. as response to a Notification
- **307 Temporary Redirect / 308 Permanent Redirect**
for HTTP-redirections (note that the request may not be automatically retransmitted)

- **405 Method Not Allowed**

If HTTP GET is not supported: for all HTTP GET requests

If HTTP GET is supported : if the client tries to call a non-safe/non-idempotent method via HTTP GET

- **415 Unsupported Media Type**

if the Content-Type is not application/json

- **Others**

for HTTP-errors or HTTP-features outside of the scope of JSON-RPC

```
//Example on success :
HTTP/1.1 200 OK
Connection : close
Content-Length: ...
Content-Type: application/json
Date : Mon, 21 Nov 2016 16:07:08 GMT
{
  "jsonrpc" : "2.0",
  "result" : 102,
  "id" : 5
}

//Example on error :
HTTP/1.1 200 OK
Connection : close
Content-Length: ...
Content-Type: application/json
Date: Mon, 21 Nov 2016 16:10:05 GMT
{
  "jsonrpc" : "2.0",
  "error" : {
    "code" : -32600,
    "message" : "Invalid Request.",
    "data" : "'method' is missing"
  }, "id" : 6
}
```

Syntax:

```
--> data sent to TV IPControl Server
<-- data sent to CI Client
```

rpc call with positional parameters:

```
--> {"jsonrpc": "2.0", "method": "subtract", "params": [42, 23], "id": 1}
<-- {"jsonrpc": "2.0", "result": 19, "id": 1}

--> {"jsonrpc": "2.0", "method": "subtract", "params": [23, 42], "id": 2}
<-- {"jsonrpc": "2.0", "result": -19, "id": 2}
```

rpc call with named parameters:

```
--> {"jsonrpc": "2.0", "method": "subtract", "params": {"subtrahend": 23, "minuend": 42}, "id": 3}
<-- {"jsonrpc": "2.0", "result": 19, "id": 3}

--> {"jsonrpc": "2.0", "method": "subtract", "params": {"minuend": 42, "subtrahend": 23}, "id": 4}
<-- {"jsonrpc": "2.0", "result": 19, "id": 4}
```

rpc call of non-existent method:

```
--> {"jsonrpc": "2.0", "method": "foobar", "id": "1"}
<-- {"jsonrpc": "2.0", "error": {"code": -32601, "message": "Method not found"}, "id": "1"}
```

rpc call with invalid JSON:

```
--> {"jsonrpc": "2.0", "method": "foobar", "params": "bar", "baz]
<-- {"jsonrpc": "2.0", "error": {"code": -32700, "message": "Parse error"}, "id": null}
```

rpc call with invalid Request object:

```
--> {"jsonrpc": "2.0", "method": 1, "params": "bar"}
<-- {"jsonrpc": "2.0", "error": {"code": -32600, "message": "Invalid Request"}, "id": null}
```

rpc call with an empty Array:

```
--> []
```

```
<-- {"jsonrpc": "2.0", "error": {"code": -32600, "message": "Invalid Request"}, "id": null}
```

Note: Batch operation is not guaranteed on the Samsung IP Control server.

Samsung TV IP Control has no need to use notification.

1.6. Server Error Code for JSON-RPC

Samsung TV IP Control server provides server error codes using below JSON-RPC error code.

- json error code : -32000 to -32099 are reserved for implementation-defined server-errors

errorCode	data	meaning
-32000	Unknown	There's an unknown error on TV.
-32001	Not supported	model The executed control is not supported on current model.
-32002	Failed	The executed control is failed.
-32003	Invalid operation	The executed control has invalid data.

JSON-RPC call:

```
// --> : data sent to TV IPControl Server
// <-- : data sent to CI Client

// to control power
--> {"jsonrpc" : "2.0", "method" : "remoteKeyControl", "params" : {"powerControl" : "cursorUp"}, "id" : 1}
<-- {"jsonrpc" : "2.0", "error" : {"code" : -32003,
                                "message" : "Server error",
                                "data" : "Invalid operation"}, "id" : 1}
```


2. Reference

- [1] Samsung Multiple Display Control Protocol Ver. 13.9c
- [2] JSON-RPC 2.0 specification <http://www.jsonrpc.org/specification>
- [3] HTTP over TLS <https://en.wikipedia.org/wiki/HTTPS>
- [4] TLS 1.2 <https://tools.ietf.org/html/rfc5246>
- [5] UPnP Device Architecture v1.0, July, 2006 <http://www.upnp.org>

3. Control

3.1. TV Control List

Control	Method #	Method	Params(for data set)	RW	Value
Power	1	powerControl	YES	RW	String
Volume	2	directVolumeControl	YES	RW	0 - 100
Volume	3	volumeUpDnControl	YES	W	String
Mute	4	muteControl	YES	RW	String
Channel	5	directChannelControl	YES	RW	Multi Param
Channel	6	channelUpDnControl	YES	W	String
Source	7	inputSourceControl	YES	RW	String
Source	8	USBSourceControl	YES	RW	Multi Param
Source	9	RVUSourceControl [Deprecated]	YES	RW	Multi Param
Picture	10	pictureModeControl	YES	RW	String
Picture	11	contrastControl	YES	RW	0 - 50
Picture	12	brightnessControl	YES	RW	(-5) - 5
Picture	13	sharpnessControl	YES	RW	0 - 20
Picture	14	colorControl	YES	RW	0 - 50
Picture	15	tintControl	YES	RW	(-15) - (15)
Picture	16	pictureSizeControl	YES	RW	String
Picture	17	pictureCalibrationModeControl [Deprecated]	YES	RW	String
Picture	18	backlightControl	YES	RW	0 - 50
Picture	19	digitalCleanViewControl	YES	RW	String
Picture	20	autoMotionPlusControl	YES	RW	String
Picture	21	AMP.BlurReductionControl	YES	RW	0 - 10
Picture	22	AMP.JudderReductionControl	YES	RW	0 - 10
Picture	23	AMP.LEDClearMotionControl	YES	RW	String
Picture	24	localDimmingControl	YES	RW	String
Picture	25	filmModeControl	YES	RW	String
Picture	26	constrastEnhancerControl	YES	RW	String
Picture	27	colorToneControl	YES	RW	String

Control	Method #	Method	Params(for data set)	RW	Value
Picture	28	WB2PointControl	YES	RW	Multi Param
Picture	29	WB20PointModeControl	YES	RW	String
Picture	30	WB20P.IntervalControl	YES	RW	String
Picture	31	WB20P.RedControl	YES	RW	(-50) – 50
Picture	32	WB20P.GreenControl	YES	RW	(-50) – 50
Picture	33	WB20P.BlueControl	YES	RW	(-50) – 50
Picture	34	gammaModeControl	YES	RW	String
Picture	35	gamma.BT1886Control	YES	RW	(-3) – 3
Picture	36	gamma.ST2084Control	YES	RW	(-3) – 3
Picture	37	gamma.HLGControl	YES	RW	(-3) – 3
Picture	38	RGBOnlyModeControl [Deprecated]	YES	RW	String
Picture	39	colorSpaceControl	YES	RW	String
Picture	40	colorSpace.ColorControl	YES	RW	String
Picture	41	colorSpace.ColorAdjustmentPointControl	YES	RW	String
Picture	42	colorSpace.RedControl	YES	RW	0 – 100
Picture	43	colorSpace.GreenControl	YES	RW	0 – 100
Picture	44	colorSpace.BlueControl	YES	RW	0 – 100
Sound	45	soundModeControl	YES	RW	String
Sound	46	speakerSelectControl	YES	RW	String
Sound	47	externalSpeakerControl	YES	RW	Multi Param
Status	48	getTVStates	NO	R	Multi Param
Status	49	getVideoStates	NO	R	Multi Param
Remote Key	50	remoteKeyControl	YES	W	String
App	51	directAccessControl	YES	RW	Multi Param
App	52	firstScreenAppControl	YES	RW	String
App	53	multiviewControl	YES	RW	String
ArtMode	54	artModeControl	YES	RW	String
DisplayRotator	55	displayRotatorControl	YES	RW	String

Note: Power > powerControl > powerOn method for powerOff state: either WoL/WoW or powerOn option could be used. Please check powerControl part.

3.1.1. Power Control

Function: controls TV power.

Method	Params(for data set)	Value
powerControl	power	powerOff
powerControl	power	powerOn or Use WoL/WoW
powerControl	power	Reboot

JSON-RPC call:

```
// --> : data sent to TV IPControl Server
// <-- : data sent to CI Client

// to get the power state
--> {"jsonrpc" : "2.0", "method" : "powerControl", "id" : 1}
<-- {"jsonrpc" : "2.0", "result" : {"power" : "powerOn"}, "id" : 1}

// to control power
--> {"jsonrpc" : "2.0", "method" : "powerControl", "params" : {"power" : "powerOff"}, "id" : 1}
<-- {"jsonrpc" : "2.0", "result" : {"power" : "powerOff"}, "id" : 1}
```

Note: WoL/WoW magic packets can be transmitted to TV instead of the client sending "powerOn" when the TV is in poweroff state. But, if IPControlServer is still responsive with "powerOff" state, client can send "powerOn" message for screen on because WoL/WoW is not acceptable by hardware level for this case.

Recommendation for client: client should check TV power state very carefully. WoL/WoW or powerOn option should be run properly due to TV state. (i.e: if there's no server side immediate response, client can regard TV is powered off state. Client has to send WoL for this case.) Due to wireless communication characteristic, TV should be connected by wired network for WoL support.

WoL/WoW Magic Packet Information

For WoL magic packet, client only needs to include basic magic packet like below :

- 6 bytes 0xFF magic word + 16 * 6 bytes TV Ethernet MAC address = 102 bytes.

For WoW magic packet, client can send magic packet to TV only if the wifi connection type is WPA2-PSK. TV CAN NOT guarantee WoW support due to wireless network characteristic.

3.1.2. Volume Control

Function: controls TV volume.

3.1.2.1. Direct Volume Control

Method	Params(for data set)	Value
directVolumeControl	volume	0 - 100

Note: In case of using AudioOutput/Optical, External speaker, directVolumeControl method does not work.

3.1.2.2. Volume Up/Down Control

Method [Set Only]	Params(for data set)	Value
volumeUpDnControl	control	volumeUp
volumeUpDnControl	control	volumeDn

Note: In case of using **AudioOutput/Optical, External** speaker, volumeUpDnControl method may not work.

JSON-RPC call:

```
// --> : data sent to TV IPControl Server
// <-- : data sent to CI Client

// to get the volume value
--> {"jsonrpc" : "2.0", "method" : "directVolumeControl", "id" : 1}
<-- {"jsonrpc" : "2.0", "result" : {"volume" : 10}, "id" : 1}

// to set the volume value directly
--> {"jsonrpc" : "2.0", "method" : "directVolumeControl", "params" : {"volume" : 15}, "id" : 1}
<-- {"jsonrpc" : "2.0", "result" : {"volume" : 15}, "id" : 1}

// to control volume up/down
--> {"jsonrpc" : "2.0", "method" : "volumeUpDnControl", "params" : {"control" : "volumeUp"}, "id" : 1}
<-- {"jsonrpc" : "2.0", "result" : {"control" : "volumeUp"}, "id" : 1}
```

3.1.3. Mute Control

Function: controls TV mute.

Method	Params(for data set)	Value
muteControl	mute	muteOff
muteControl	mute	muteOn

Note: In case of using **AudioOutput/Optical, External** speaker, muteControl method may not work.

JSON-RPC call:

```
// --> : data sent to TV IPControl Server
// <-- : data sent to CI Client

// to get the mute state
```

```
--> {"jsonrpc" : "2.0", "method" : "muteControl", "id" : 1}
<-- {"jsonrpc" : "2.0", "result" : {"mute" : "muteOff"}, "id" : 1}

// to control mute on/off
--> {"jsonrpc" : "2.0", "method" : "muteControl", "params" : "muteOn", "id" : 1}
<-- {"jsonrpc" : "2.0", "result" : {"mute" : "muteOn"}, "id" : 1}
```

3.1.4. Channel Control

Function: controls TV channel.

3.1.4.1. Direct Channel Control

Method	Params(for data set)	Value
directChannelControl	atvDtv	atv
directChannelControl	atvDtv	dtv
directChannelControl	atvDtv	tvplus*
directChannelControl	atvDtv	iptv(set only)
directChannelControl	airCable	air
directChannelControl	airCable	cable
directChannelControl	channelNum	*various
		ATV : 1 ~ 135
		DTV : 0 ~ 999
		Minor Ch.
		0 ~ 999
		Ex1 : 10
		Ex2 : 10-5

Note: directChannelControl method running includes TV source usage. So, if user was using TV source before any application launch, source will be changed to TV if this method is run. For the other sources(HDMI, COMPONENT, AV, RVU(Deprecated)), directChannelControl method is not available.

And atvDtv, airCable options are mandatory for TV system since all those informations are needed for directChannelControl working.

Note: "tvplus" is supported from 2022 products. Any value of airCable for tvplus channel switching will be ignored internally (client SHOULD send airCable value).

Note: In using iptv. select airCable as none

3.1.4.2. Channel Up/Down Control

Method [Set Only]	Params(for data set)	Value
channelUpDnControl	control	channelUp

Method [Set Only]	Params(for data set)	Value
channelUpDnControl	control	channelDn

Note: depending on the TV source type or mode, channelUpDn operation may not work.

JSON-RPC call:

```
// --> : data sent to TV IPControl Server
// <-- : data sent to CI Client

// to get the channel value
--> {"jsonrpc" : "2.0", "method" : "directChannelControl", "id" : 1}
<-- {"jsonrpc" : "2.0", "result" : { "atvDtv" : "dtv",
                                   "airCable" : "cable",
                                   "channelNum" : "10-5"}, "id" : 1}

// to set the channel value directly
--> {"jsonrpc" : "2.0", "method" : "directChannelControl", "params" : {"atvDtv" : "dtv",
                                                                    "airCable" : "cable",
                                                                    "channelNum" : "20"}, "id" :
1}
<-- {"jsonrpc" : "2.0", "result" : {"atvDtv" : "dtv",
                                   "airCable" : "cable",
                                   "channelNum" : "20"}, "id" : 1}

// to set tvplus channel directly
--> {"jsonrpc" : "2.0", "method" : "directChannelControl", "params" : {"atvDtv" : "tvplus",
                                                                    "airCable" : "cable",
                                                                    "channelNum" : "1005"}, "id" :
1}
<-- {"jsonrpc" : "2.0", "result" : {"atvDtv" : "tvplus",
                                   "airCable" : "cable",
                                   "channelNum" : "1005"}, "id" : 1}

// to control channel up/down
--> {"jsonrpc" : "2.0", "method" : "channelUpDnControl", "params" : {"control" : "channelUp"}, "id" :
1}
<-- {"jsonrpc" : "2.0", "result" : {"control" : "channelUp"}, "id" : 1}
```

3.1.5. Source Control

3.1.5.1. Input Source Control

Function: controls TV normal input source.

Method	Params(for data set)	Value
inputSourceControl	inputSource	TV
inputSourceControl	inputSource	HDMI*(HDMI1,HDMI2,...)

Method	Params(for data set)	Value
inputSourceControl	inputSource	AV1
inputSourceControl	inputSource	COMPONENT1
inputSourceControl	inputSource	USB*
inputSourceControl	inputSource	RVU* [Deprecated]

Note : Changing TV input source is up to TV menu policy. Set operation does not work if the source is not available for TV IPControl server.

For **USB/RVU** source control, use **USBSourceControl/RVUSourceControl**. This method only notifies the current source name for USB/RVU source case(set operation does not work since there is USB/RVU device list).

HDMI4 value works for HDBaseT port on the Samsung Terrace series.

Note : HDMI ports are supported by the number of physical ports.

JSON-RPC call:

```
// --> : data sent to TV IPControl Server
// <-- : data sent to CI Client

// to get the input source value
--> {"jsonrpc" : "2.0", "method" : "inputSourceControl", "id" : 1}
<-- {"jsonrpc" : "2.0", "result" : {"inputSource" : "TV"}, "id" : 1}

// to control input source
--> {"jsonrpc" : "2.0", "method" : "inputSourceControl", "params" : {"inputSource" : "HDMI1"}, "id" : 1}
<-- {"jsonrpc" : "2.0", "result" : {"inputSource" : "HDMI1"}, "id" : 1}
```

3.1.5.2. USB Source Control

Function: controls TV USB source.

Method	Params(for data set)	Value
USBSourceControl	deviceName	string
USBSourceControl	deviceId	string

Note: Client device must get deviceList first to change TV source to USB.

JSON-RPC call:

```
// --> : data sent to TV IPControl Server
// <-- : data sent to CI Client

// to get the USB source value
--> {"jsonrpc" : "2.0", "method" : "USBSourceControl", "id" : 1}
```



```

<-- {"jsonrpc" : "2.0", "result" : [{ "deviceName" : "Samsung", "deviceId" : "ABEKSD1001"},
                                     { "deviceName" : "SanDisk", "deviceId" : "JSHDYW0012"}], "id" : 1}

// to control USB source
--> {"jsonrpc" : "2.0", "method" : "USBSourceControl",
    "params" : { "deviceName" : "SanDisk", "deviceId" : "JSHDYW0012"}, "id" : 1}
<-- {"jsonrpc" : "2.0",
    "result" : { "deviceName" : "SanDisk", "deviceId" : "JSHDYW0012"}, "id" : 1}

```

3.1.5.3. RVU Source Control [Deprecated from 2018 TV]

Function: controls TV RVU source.

Method	Params(for data set)	Value
RVUSourceControl	deviceName	string
RVUSourceControl	deviceId	string

Note: Client device must get deviceList first to change TV source to RVU. [Deprecated from 2018 TV]

JSON-RPC call:

```

// --> : data sent to TV IPControl Server
// <-- : data sent to CI Client

// to get the RVU source value
--> {"jsonrpc" : "2.0", "method" : "RVUSourceControl", "id" : 1}
<-- {"jsonrpc" : "2.0", "result" :
    [{ "deviceName" : "RVU Alliance",
        "deviceId" : "uuid:RVU Alliance-RVU-RUI-Server1_0-RID-36071022374+eth0"},
      { "deviceName" : "RVU Alliance",
        "deviceId" : "uuid:RVU Alliance-RVU-RUI-Server1_0-RID-36071022375+eth0"}],
    "id" : 1}

// to control RVU source
--> {"jsonrpc" : "2.0", "method" : "RVUSourceControl",
    "params" : { "deviceName" : "RVU Alliance",
        "deviceId" : "uuid:RVU Alliance-RVU-RUI-Server1_0-RID-36071022374+eth0"},
    "id" : 1}
<-- {"jsonrpc" : "2.0",
    "result" : { "deviceName" : "RVU Alliance",
        "deviceId" : "uuid:RVU Alliance-RVU-RUI-Server1_0-RID-36071022374+eth0"},
    "id" : 1}

```

3.1.6. Picture Control

Function: controls TV picture options.

Control	Method#	Method Name	Picture Control Type	Support Year
Picture	11	contrastControl	Basic	All
Picture	10	pictureModeControl	Basic	All
Picture	12	brightnessControl	Basic	All
Picture	13	sharpnessControl	Basic	All
Picture	14	colorControl	Basic	All
Picture	15	tintControl	Basic	All
Picture	16	pictureSizeControl	Basic	All
Picture	17	pictureCalibrationModeControl [Deprecated]	Expert	2019 only
Picture	18	backlightControl	Expert	From 2019
Picture	19	digitalCleanViewControl	Expert	From 2019
Picture	20	autoMotionPlusControl	Expert	From 2019
Picture	21	AMP.BlurReductionControl	Expert	From 2019
Picture	22	AMP.JudderReductionControl	Expert	From 2019
Picture	23	AMP.LEDClearMotionControl	Expert	From 2019
Picture	24	localDimmingControl	Expert	From 2019
Picture	25	filmModeControl	Expert	From 2019
Picture	26	constrastEnhancerControl	Expert	From 2019
Picture	27	colorToneControl	Expert	From 2019
Picture	28	WB2PointControl	Expert	From 2019
Picture	29	WB20PointModeControl	Expert	From 2019
Picture	30	WB20P.IntervalControl	Expert	From 2019
Picture	31	WB20P.RedControl	Expert	From 2019
Picture	32	WB20P.GreenControl	Expert	From 2019
Picture	33	WB20P.BlueControl	Expert	From 2019
Picture	34	gammaModeControl	Expert	From 2019
Picture	35	gamma.BT1886Control	Expert	From 2019
Picture	36	gamma.ST2084Control	Expert	From 2019
Picture	37	gamma.HLGControl	Expert	From 2019
Picture	38	RGBOnlyModeControl [Deprecated]	Expert	From 2019
Picture	39	colorSpaceControl	Expert	From 2019

Control	Method#	Method Name	Picture Control Type	Support Year
Picture	40	colorSpace.ColorControl	Expert	From 2019
Picture	41	colorSpace.ColorAdjustmentPointControl	Expert	From 2019
Picture	42	colorSpace.RedControl	Expert	From 2019
Picture	43	colorSpace.GreenControl	Expert	From 2019
Picture	44	colorSpace.BlueControl	Expert	From 2019

3.1.6.1. Picture Mode Control

Method	Params(for data set)	Value
pictureModeControl	pictureMode	Dynamic
pictureModeControl	pictureMode	Standard
pictureModeControl	pictureMode	Movie
pictureModeControl	pictureMode	Natural [Deprecated from 2022 TV]
pictureModeControl	pictureMode	FilmmakerMode [From 2020 TV]
pictureModeControl	pictureMode	*CAL-NIGHT
pictureModeControl	pictureMode	*CAL-DAY

Note: depending on the TV source type or mode, set operation may not work.

- CAL-NIGHT, CAL-DAY options can be available for picture quality experts. To enable these options, user should be able to turn on Factory option in the TV
- FilmmakerMode option is available from 2020 TV with FMM contents. Please check the Filmmaker Mode of Samsung TV.

[Natural option is NOT AVAILABLE from 2022 TV]

JSON-RPC call:

```
// --> : data sent to TV IPControl Server
// <-- : data sent to CI Client

// to get the picture mode value
--> {"jsonrpc" : "2.0", "method" : "pictureModeControl", "id" : 1}
<-- {"jsonrpc" : "2.0", "result" : {"pictureMode" : "Dynamic"}, "id" : 1}

// to control picture mode
--> {"jsonrpc" : "2.0", "method" : "pictureModeControl", "params" : {"pictureMode" : "Movie"}, "id" : 1}
<-- {"jsonrpc" : "2.0", "result" : {"pictureMode" : "Movie"}, "id" : 1}
```

3.1.6.2. Contrast Control

Method	Params(for data set)	Value
contrastControl	contrast	0 - 50

Note: depending on the TV source type or mode, set operation may not work.

Note: above 24 year model, maximum is 100 on AoT model.

JSON-RPC call:

```
// --> : data sent to TV IPControl Server
// <-- : data sent to CI Client

// to get the contrast value
--> {"jsonrpc" : "2.0", "method" : "contrastControl", "id" : 1}
<-- {"jsonrpc" : "2.0", "result" : {"contrast" : 50}, "id" : 1}

// to set the contrast value
--> {"jsonrpc" : "2.0", "method" : "contrastControl", "params" : {"contrast" : 30}, "id" : 1}
<-- {"jsonrpc" : "2.0", "result" : {"contrast" : 30}, "id" : 1}
```

3.1.6.3. Brightness Control

Method	Params(for data set)	Value
brightnessControl	brightness	(-5) – 5

Note: depending on the TV source type or mode, set operation may not work. Brightness range is -5 to 5.
TV menu name is changed from "Brightness" to "Shadow Detail" on 2020 TV (Please check the Appendix 5.2).

JSON-RPC call:

```
// --> : data sent to TV IPControl Server
// <-- : data sent to CI Client

// to get the brightness value
--> {"jsonrpc" : "2.0", "method" : "brightnessControl", "id" : 1}
<-- {"jsonrpc" : "2.0", "result" : {"brightness" : 5}, "id" : 1}

// to set the brightness value
--> {"jsonrpc" : "2.0", "method" : "brightnessControl", "params" : {"brightness" : 2}, "id" : 1}
<-- {"jsonrpc" : "2.0", "result" : {"brightness" : 2}, "id" : 1}
```

3.1.6.4. Sharpness Control

Method	Params(for data set)	Value
sharpnessControl	sharpness	0 - 20

Note: depending on the TV source type or mode, set operation may not work.

JSON-RPC call:

```
// --> : data sent to TV IPControl Server
// <-- : data sent to CI Client

// to get the sharpness value
--> {"jsonrpc" : "2.0", "method" : "sharpnessControl", "id" : 1}
<-- {"jsonrpc" : "2.0", "result" : {"sharpness" : 10}, "id" : 1}

// to set the sharpness value
--> {"jsonrpc" : "2.0", "method" : "sharpnessControl", "params" : {"sharpness" : 20}, "id" : 1}
<-- {"jsonrpc" : "2.0", "result" : {"sharpness" : 20}, "id" : 1}
```

3.1.6.5. Color Control

Method	Params(for data set)	Value
colorControl	color	0 - 50

Note: depending on the TV source type or mode, set operation may not work.

JSON-RPC call:

```
// --> : data sent to TV IPControl Server
// <-- : data sent to CI Client

// to get the color value
--> {"jsonrpc" : "2.0", "method" : "colorControl", "id" : 1}
<-- {"jsonrpc" : "2.0", "result" : {"color" : 50}, "id" : 1}

// to set the color value
--> {"jsonrpc" : "2.0", "method" : "colorControl", "params" : {"color" : 25}, "id" : 1}
<-- {"jsonrpc" : "2.0", "result" : {"color" : 25}, "id" : 1}
```

3.1.6.6. Tint Control

Method	Params(for data set)	Value
tintControl	tint	(-15) - 15

Note: depending on the TV source type or mode, set operation may not work.
The value of the tint means R value of tint control. For example, TV will response with 15 if current tint setting is "R15". TV will response with -15 if current tint setting is "G15".

JSON-RPC call:

```
// --> : data sent to TV IPControl Server
// <-- : data sent to CI Client
```

```
// to get the tint value
--> {"jsonrpc" : "2.0", "method" : "tintControl", "id" : 1}
<-- {"jsonrpc" : "2.0", "result" : {"tint" : 0}, "id" : 1}

// to set the tint value
--> {"jsonrpc" : "2.0", "method" : "tintControl", "params" : {"tint" : 15}, "id" : 1}
<-- {"jsonrpc" : "2.0", "result" : {"tint" : 15}, "id" : 1}
```

3.1.6.7. Picture Size Control

Method	Params(for data set)	Value
pictureSizeControl	pictureSize	16:9
pictureSizeControl	pictureSize	4:3

Note: depending on the TV source type or mode, set operation may not work.

JSON-RPC call:

```
// --> : data sent to TV IPControl Server
// <-- : data sent to CI Client

// to get the picture size value
--> {"jsonrpc" : "2.0", "method" : "pictureSizeControl", "id" : 1}
<-- {"jsonrpc" : "2.0", "result" : {"pictureSize" : "16:9"}, "id" : 1}

// to control picture size
--> {"jsonrpc" : "2.0", "method" : "pictureSizeControl", "params" : {"pictureSize" : "4:3"}, "id" : 1}
<-- {"jsonrpc" : "2.0", "result" : {"pictureSize" : "4:3"}, "id" : 1}
```

3.1.6.8. Picture Calibration Mode Control [Deprecated]

Method	Params(for data set)	Value
pictureCalibrationModeControl	pictureCalibrationMode	Off
pictureCalibrationModeControl	pictureCalibrationMode	On

Note: This method is for enabling picture calibration mode in the pictureModeControl method.

- If pictureCalibrationMode is "On", pictureModeControl can have extra options (CAL-NIGHT, CAL-DAY). With these options, TV allows expert settings for picture mode.

[Deprecated]

JSON-RPC call:

```
// --> : data sent to TV IPControl Server
// <-- : data sent to CI Client

// to get the picture calibration mode
--> {"jsonrpc" : "2.0", "method" : "pictureCalibrationModeControl", "id" : 1}
<-- {"jsonrpc" : "2.0", "result" : {"pictureCalibrationMode" : "Off"}, "id" : 1}

// to set the picture calibration mode
--> {"jsonrpc" : "2.0", "method" : "pictureCalibrationModeControl", "params" :
{"pictureCalibrationMode" : "On"}, "id" : 1}
<-- {"jsonrpc" : "2.0", "result" : {"pictureCalibrationMode" : "On"}, "id" : 1}
```

3.1.6.9. Backlight Control

Method	Params(for data set)	Value
backlightControl	backlight	0 – 50

Note: depending on the TV source type or mode, set operation may not work.

TV menu name is changed from "Backlight" to "Brightness" on 2020 TV (Please check the Appendix 5.2).

JSON-RPC call:

```
// --> : data sent to TV IPControl Server
// <-- : data sent to CI Client

// to get the backlight value
--> {"jsonrpc" : "2.0", "method" : "backlightControl", "id" : 1}
<-- {"jsonrpc" : "2.0", "result" : {"backlight" : 50}, "id" : 1}

// to set the backlight value
--> {"jsonrpc" : "2.0", "method" : "backlightControl", "params" : {"backlight" : 10}, "id" : 1}
<-- {"jsonrpc" : "2.0", "result" : {"backlight" : 10}, "id" : 1}
```

3.1.6.10. Digital Clean View Control

Method	Params(for data set)	Value
digitalCleanViewControl	digitalCleanView	Auto
digitalCleanViewControl	digitalCleanView	*Low [Deprecated]
digitalCleanViewControl	digitalCleanView	Off

- Low option IS NOT available from 2020 TV.

Digital Clean View is renamed to Noise Reduction on 2020 TV menu. (Appendix 5.1).

JSON-RPC call:

```
// --> : data sent to TV IPControl Server
// <-- : data sent to CI Client

// to get the digital clean view mode value
--> {"jsonrpc" : "2.0", "method" : "digitalCleanViewControl", "id" : 1}
<-- {"jsonrpc" : "2.0", "result" : {"digitalCleanView" : "Auto"}, "id" : 1}

// to control digital clean view mode
--> {"jsonrpc" : "2.0", "method" : "digitalCleanViewControl", "params" : {"digitalCleanView" : "Low"},
"id" : 1}
<-- {"jsonrpc" : "2.0", "result" : {"digitalCleanView" : "Low"}, "id" : 1}
```

3.1.6.11. Auto Motion Plus Control

Method	Params(for data set)	Value
autoMotionPlusControl	autoMotionPlus	Off
autoMotionPlusControl	autoMotionPlus	Auto
autoMotionPlusControl	autoMotionPlus	Custom

Note: Below three sub options are available only if autoMotionPlusControl value is set as "Custom".

- Blur Reduction (120Hz panel only)
- Judder Reduction
- LED Clear Motions

Auto Motion Plus is renamed to "Picture Clarity" on 2020 TV menu. (Appendix 5.1)

JSON-RPC call:

```
// --> : data sent to TV IPControl Server
// <-- : data sent to CI Client

// to get the auto motion plus mode value
--> {"jsonrpc" : "2.0", "method" : "autoMotionPlusControl", "id" : 1}
<-- {"jsonrpc" : "2.0", "result" : {"autoMotionPlus" : "Auto"}, "id" : 1}

// to control auto motion plus mode
--> {"jsonrpc" : "2.0", "method" : "autoMotionPlusControl", "params" : {"autoMotionPlus" : "Custom"},
"id" : 1}
<-- {"jsonrpc" : "2.0", "result" : {"autoMotionPlus" : "Custom"}, "id" : 1}
```

3.1.6.12. Blur Reduction Control [available with Auto Motion Plus : Custom, 120Hz Panel]

Method	Params(for data set)	Value
AMP.blurReductionControl	AMP.blurReduction	0 – 10

Note: This sub option is available only if autoMotionPlusControl value is set as "Custom". And TV panel type should support 120Hz refresh rate.

Auto Motion Plus is renamed to "Picture Clarity" on 2020 TV menu. (Appendix 5.1)

JSON-RPC call:

```
// --> : data sent to TV IPControl Server
// <-- : data sent to CI Client

// to get the auto motion plus mode , blurReduction mode value
--> {"jsonrpc" : "2.0", "method" : "autoMotionPlusControl", "id" : 1}
<-- {"jsonrpc" : "2.0", "result" : {"autoMotionPlus" : "Custom"}, "id" : 1}
--> {"jsonrpc" : "2.0", "method" : "AMP.blurReductionControl", "id" : 1}
<-- {"jsonrpc" : "2.0", "result" : {"AMP.blurReduction" : "10"}, "id" : 1}

// to control blur reduction mode
--> {"jsonrpc" : "2.0", "method" : "AMP.blurReductionControl", "params" : {"AMP.blurReduction" : "10"}, "id" : 1}
<-- {"jsonrpc" : "2.0", "result" : {"AMP.blurReduction" : "10"}, "id" : 1}
```

3.1.6.13. Judder Reduction Control [available with Auto Motion Plus : Custom]

Method	Params(for data set)	Value
AMP.judderReductionControl	AMP.judderReduction	0 – 10

Note: This sub option is available only if autoMotionPlusControl value is set as "Custom".

Auto Motion Plus is renamed to "Picture Clarity" on 2020 TV menu. (Appendix 5.1)

JSON-RPC call:

```
// --> : data sent to TV IPControl Server
// <-- : data sent to CI Client

// to get the auto motion plus mode , judderReduction mode value
--> {"jsonrpc" : "2.0", "method" : "autoMotionPlusControl", "id" : 1}
<-- {"jsonrpc" : "2.0", "result" : {"autoMotionPlus" : "Custom"}, "id" : 1}
--> {"jsonrpc" : "2.0", "method" : "AMP.judderReductionControl", "id" : 1}
<-- {"jsonrpc" : "2.0", "result" : {"AMP.judderReduction" : "10"}, "id" : 1}

// to control judder reduction mode
--> {"jsonrpc" : "2.0", "method" : "AMP.judderReductionControl", "params" : {"AMP.judderReduction" : "5"}, "id" : 1}
<-- {"jsonrpc" : "2.0", "result" : {"AMP.judderReduction" : "5"}, "id" : 1}
```

3.1.6.14. LED Clear Motion Control [available with Auto Motion Plus : Custom]

Method	Params(for data set)	Value
AMP.LEDClearMotionControl	AMP.LEDClearMotion	Off
AMP.LEDClearMotionControl	AMP.LEDClearMotion	On

Note: This sub option is available only if autoMotionPlusControl value is set as "Custom".

Auto Motion Plus is renamed to "Picture Clarity" on 2020 TV menu. (Appendix 5.1)

JSON-RPC call:

```
// --> : data sent to TV IPControl Server
// <-- : data sent to CI Client

// to get the auto motion plus mode , LED clear motion mode value
--> {"jsonrpc" : "2.0", "method" : "autoMotionPlusControl", "id" : 1}
<-- {"jsonrpc" : "2.0", "result" : {"autoMotionPlus" : "Custom"}, "id" : 1}
--> {"jsonrpc" : "2.0", "method" : "AMP.LEDClearMotionControl", "id" : 1}
<-- {"jsonrpc" : "2.0", "result" : {"AMP.LEDClearMotion" : "On"}, "id" : 1}

// to control LED clear motion mode
--> {"jsonrpc" : "2.0", "method" : "AMP.LEDClearMotionControl", "params" : {"AMP.LEDClearMotion" : "Off"}, "id" : 1}
<-- {"jsonrpc" : "2.0", "result" : {"AMP.LEDClearMotion" : "Off"}, "id" : 1}
```

3.1.6.15. Local Dimming Control

Method	Params(for data set)	Value
localDimmingControl	localDimming	Low
localDimmingControl	localDimming	Standard
localDimmingControl	localDimming	High

JSON-RPC call:

```
// --> : data sent to TV IPControl Server
// <-- : data sent to CI Client

// to get the local dimming mode value
--> {"jsonrpc" : "2.0", "method" : "localDimmingControl", "id" : 1}
<-- {"jsonrpc" : "2.0", "result" : {"localDimming" : "High"}, "id" : 1}

// to control local dimming mode
--> {"jsonrpc" : "2.0", "method" : "localDimmingControl", "params" : {"localDimming" : "Low"}, "id" : 1}
<-- {"jsonrpc" : "2.0", "result" : {"localDimming" : "Low"}, "id" : 1}
```

3.1.6.16. Film Mode Control

Method	Params(for data set)	Value
filmModeControl	filmMode	Off
filmModeControl	filmMode	Auto1
filmModeControl	filmMode	Auto2

Note: This option is available on interlace mode only. On the progress mode, this mode is not controllable.

Auto1, Auto2 options will be treated as Auto from 2020 TV since there's only Auto option in TV menu.
(Appendix 5.1)

Client CAN send filmModeControl any of above params. TV will treat it as Auto.

JSON-RPC call:

```
// --> : data sent to TV IPControl Server
// <-- : data sent to CI Client

// to get the film mode value
--> {"jsonrpc" : "2.0", "method" : "filmModeControl", "id" : 1}
<-- {"jsonrpc" : "2.0", "result" : {"filmMode" : "Auto1"}, "id" : 1}

// to control flim mode
--> {"jsonrpc" : "2.0", "method" : "filmModeControl", "params" : {"filmMode" : "Auto2"}, "id" : 1}
<-- {"jsonrpc" : "2.0", "result" : {"filmMode" : "Auto2"}, "id" : 1}
```

3.1.6.17. Contrast Enhancer Control

Method	Params(for data set)	Value
contrastEnhancerControl	contrastEnhancer	Off
contrastEnhancerControl	contrastEnhancer	Low
contrastEnhancerControl	contrastEnhancer	High

JSON-RPC call:

```
// --> : data sent to TV IPControl Server
// <-- : data sent to CI Client

// to get the contrast enhancer mode value
--> {"jsonrpc" : "2.0", "method" : "contrastEnhancerControl", "id" : 1}
<-- {"jsonrpc" : "2.0", "result" : {"contrastEnhancer" : "Off"}, "id" : 1}

// to control contrast enhancer mode
--> {"jsonrpc" : "2.0", "method" : "contrastEnhancerControl", "params" : {"contrastEnhancer" : "Low"}, "id" : 1}
<-- {"jsonrpc" : "2.0", "result" : {"contrastEnhancer" : "Low"}, "id" : 1}
```

3.1.6.18. Color Tone Control

Method	Params(for data set)	Value
colorToneControl	colorTone	Cool
colorToneControl	colorTone	Standard
colorToneControl	colorTone	Warm1
colorToneControl	colorTone	Warm2

JSON-RPC call:

```
// --> : data sent to TV IPControl Server
// <-- : data sent to CI Client

// to get the color tone value
--> {"jsonrpc" : "2.0", "method" : "colorToneControl", "id" : 1}
<-- {"jsonrpc" : "2.0", "result" : {"colorTone" : "Standard"}, "id" : 1}

// to control color tone
--> {"jsonrpc" : "2.0", "method" : "colorToneControl", "params" : {"colorTone" : "Warm1"}, "id" : 1}
<-- {"jsonrpc" : "2.0", "result" : {"colorTone" : "Warm1"}, "id" : 1}
```

3.1.6.19. White Balance 2Point Control

Method	Params(for data set)	Value
WB2PointControl	R-Gain	(-50) – 50
WB2PointControl	G-Gain	(-50) – 50
WB2PointControl	B-Gain	(-50) – 50
WB2PointControl	R-Offset	(-50) – 50
WB2PointControl	G-Offset	(-50) – 50
WB2PointControl	B-Offset	(-50) – 50

JSON-RPC call:

```
// --> : data sent to TV IPControl Server
// <-- : data sent to CI Client

// to get white balance 2 point information
--> {"jsonrpc" : "2.0", "method" : "WB2PointControl", "id" : 1}
<-- {"jsonrpc" : "2.0", "result" : {"R-Gain" : 0,
                                   "G-Gain" : 0,
                                   "B-Gain" : 0,
                                   "R-Offset" : 0,
                                   "G-Offset" : 0,
```

```

        "B-Offset" : 0}, "id" : 1}

// to control white balance 2 point values
--> {"jsonrpc" : "2.0", "method" : "WB2PointControl", "params" : {"R-Gain" : 1,
                                                                    "G-Gain" : 1,
                                                                    "B-Gain" : 1,
                                                                    "R-Offset" : 0,
                                                                    "G-Offset" : 0,
                                                                    "B-Offset" : 0}, "id" : 1}

<-- {"jsonrpc" : "2.0", "result" : {"R-Gain" : 1,
                                    "G-Gain": 1,
                                    "B-Gain" : 1,
                                    "R-Offset" : 0,
                                    "G-Offset" : 0,
                                    "B-Offset" : 0}, "id" : 1}

```

3.1.6.20. White Balance 20Point Mode Control

Method	Params(for data set)	Value
WB20PointModeControl	WB20PointMode	Off
WB20PointModeControl	WB20PointMode	On

Note: Below options are available only if WB20PointMode value is "On".

- WB20P.Interval
- WB20P.Red
- WB20P.Green
- WB20P.Blue

JSON-RPC call:

```

// --> : data sent to TV IPControl Server
// <-- : data sent to CI Client

// to get the white balance 20 point mode
--> {"jsonrpc" : "2.0", "method" : "WB20PointModeControl", "id" : 1}
<-- {"jsonrpc" : "2.0", "result" : {"WB20PointMode" : "On"}, "id" : 1}

// to control white balance 20 point mode
--> {"jsonrpc" : "2.0", "method" : "WB20PointModeControl", "params" : {"WB20PointMode" : "Off"}, "id" : 1}
<-- {"jsonrpc" : "2.0", "result" : {"WB20PointMode" : "Off"}, "id" : 1}

```

3.1.6.21. WB20P.Interval Control

Method	Params(for data set)	Value
WB20P.IntervalControl	WB20P.Interval	5%

Method	Params(for data set)	Value
WB20P.IntervalControl	WB20P.Interval	10%
WB20P.IntervalControl	WB20P.Interval	15%
WB20P.IntervalControl	WB20P.Interval	20%
WB20P.IntervalControl	WB20P.Interval	25%
WB20P.IntervalControl	WB20P.Interval	30%
WB20P.IntervalControl	WB20P.Interval	35%
WB20P.IntervalControl	WB20P.Interval	40%
WB20P.IntervalControl	WB20P.Interval	45%
WB20P.IntervalControl	WB20P.Interval	50%
WB20P.IntervalControl	WB20P.Interval	55%
WB20P.IntervalControl	WB20P.Interval	60%
WB20P.IntervalControl	WB20P.Interval	65%
WB20P.IntervalControl	WB20P.Interval	70%
WB20P.IntervalControl	WB20P.Interval	75%
WB20P.IntervalControl	WB20P.Interval	80%
WB20P.IntervalControl	WB20P.Interval	85%
WB20P.IntervalControl	WB20P.Interval	90%
WB20P.IntervalControl	WB20P.Interval	95%
WB20P.IntervalControl	WB20P.Interval	100%

Note: This option is available only if WB20PointMode value is "On".

JSON-RPC call:

```
// --> : data sent to TV IPControl Server
// <-- : data sent to CI Client

// to get the white balance 20 point interval value
--> {"jsonrpc" : "2.0", "method" : "WB20PointControl", "id" : 1}
<-- {"jsonrpc" : "2.0", "result" : {"WB20Point" : "On"}, "id" : 1}
--> {"jsonrpc" : "2.0", "method" : "WB20P.IntervalControl", "id" : 1}
<-- {"jsonrpc" : "2.0", "result" : {"WB20P.Interval" : "100%"}, "id" : 1}

// to control white balance 20 point interval value
--> {"jsonrpc" : "2.0", "method" : "WB20P.IntervalControl", "params" : {"WB20P.Interval" : "50%"},
"id" : 1}
<-- {"jsonrpc" : "2.0", "result" : {"WB20P.Interval" : "50%"}, "id" : 1}
```

3.1.6.22. WB20P.Red Control

Method	Params(for data set)	Value
WB20P.RedControl	WB20P.Red	(-50) – 50

Note: This option is available only if WB20PointMode value is "On".

JSON-RPC call:

```
// --> : data sent to TV IPControl Server
// <-- : data sent to CI Client

// to get the white balance 20 point red value
--> {"jsonrpc" : "2.0", "method" : "WB20PointControl", "id" : 1}
<-- {"jsonrpc" : "2.0", "result" : {"WB20Point" : "On"}, "id" : 1}
--> {"jsonrpc" : "2.0", "method" : "WB20P.RedControl", "id" : 1}
<-- {"jsonrpc" : "2.0", "result" : {"WB20P.Red" : 0}, "id" : 1}

// to control white balance 20 point red value
--> {"jsonrpc" : "2.0", "method" : "WB20P.RedControl", "params" : {"WB20P.Red" : 50}, "id" : 1}
<-- {"jsonrpc" : "2.0", "result" : {"WB20P.Red" : 50}, "id" : 1}
```

3.1.6.23. WB20P.Green Control

Method	Params(for data set)	Value
WB20P.GreenControl	WB20P.Green	(-50) – 50

Note: This option is available only if WB20PointMode value is "On".

JSON-RPC call:

```
// --> : data sent to TV IPControl Server
// <-- : data sent to CI Client

// to get the white balance 20 point green value
--> {"jsonrpc" : "2.0", "method" : "WB20PointControl", "id" : 1}
<-- {"jsonrpc" : "2.0", "result" : {"WB20Point" : "On"}, "id" : 1}
--> {"jsonrpc" : "2.0", "method" : "WB20P.GreenControl", "id" : 1}
<-- {"jsonrpc" : "2.0", "result" : {"WB20P.Green" : 0}, "id" : 1}

// to control white balance 20 point green value
--> {"jsonrpc" : "2.0", "method" : "WB20P.GreenControl", "params" : {"WB20P.Green" : 50}, "id" : 1}
<-- {"jsonrpc" : "2.0", "result" : {"WB20P.Green" : 50}, "id" : 1}
```

3.1.6.24. WB20P.Blue Control

Method	Params(for data set)	Value
WB20P.BlueControl	WB20P.Blue	(-50) – 50

Note: This option is available only if WB20PointMode value is "On".

JSON-RPC call:

```
// --> : data sent to TV IPControl Server
// <-- : data sent to CI Client

// to get the white balance 20 point blue value
--> {"jsonrpc" : "2.0", "method" : "WB20PointControl", "id" : 1}
<-- {"jsonrpc" : "2.0", "result" : {"WB20Point" : "On"}, "id" : 1}
--> {"jsonrpc" : "2.0", "method" : "WB20P.BlueControl", "id" : 1}
<-- {"jsonrpc" : "2.0", "result" : {"WB20P.Blue" : 0}, "id" : 1}

// to control white balance 20 point blue value
--> {"jsonrpc" : "2.0", "method" : "WB20P.BlueControl", "params" : {"WB20P.Blue" : 50}, "id" : 1}
<-- {"jsonrpc" : "2.0", "result" : {"WB20P.Blue" : 50}, "id" : 1}
```

3.1.6.25. Gamma Mode Control

Method	Params(for data set)	Value
gammaModeControl	gammaMode	BT.1886
gammaModeControl	gammaMode	ST.2084
gammaModeControl	gammaMode	HLG
gammaModeControl	gammaMode	2.2

Note: Depending on the gammaMode value, relevant value can be controllable using each control method.

- BT.1886 : SDR
- ST.2084 : HDR
- HLG : HLG
- 2.2 : SDR(no control option)

JSON-RPC call:

```
// --> : data sent to TV IPControl Server
// <-- : data sent to CI Client

// to get the gamma mode info
--> {"jsonrpc" : "2.0", "method" : "gammaModeControl", "id" : 1}
<-- {"jsonrpc" : "2.0", "result" : {"gammaMode" : "BT.1886"}, "id" : 1}

// to control gamma mode
```



```
--> {"jsonrpc" : "2.0", "method" : "gammaModeControl", "params" : {"gammaMode" : "ST.2084"}, "id" : 1}
<-- {"jsonrpc" : "2.0", "result" : {"gammaMode" : "ST.2084"}, "id" : 1}
```

3.1.6.26. Gamma BT.1886 Control *[available with gammaMode : BT.1886]*

Method	Params(for data set)	Value
gamma.BT1886Control	gamma.BT1886	(-3) – 3

JSON-RPC call:

```
// --> : data sent to TV IPControl Server
// <-- : data sent to CI Client

// to get the gamma BT.1886 info
--> {"jsonrpc" : "2.0", "method" : "gammaModeControl", "id" : 1}
<-- {"jsonrpc" : "2.0", "result" : {"gammaMode" : "BT.1886"}, "id" : 1}
--> {"jsonrpc" : "2.0", "method" : "gamma.BT1886Control", "id" : 1}
<-- {"jsonrpc" : "2.0", "result" : {"gamma.BT1886" : 0}, "id" : 1}

// to control gamma BT.1886
--> {"jsonrpc" : "2.0", "method" : "gamma.BT1886Control", "params" : {"gamma.BT1886" : 1}, "id" : 1}
<-- {"jsonrpc" : "2.0", "result" : {"gamma.BT1886" : 1}, "id" : 1}
```

3.1.6.27. Gamma ST.2084 Control *[available with gammaMode : ST.2084]*

Method	Params(for data set)	Value
gamma.ST2084Control	gamma.ST2084	(-3) – 3

JSON-RPC call:

```
// --> : data sent to TV IPControl Server
// <-- : data sent to CI Client

// to get the gamma ST.2084 info
--> {"jsonrpc" : "2.0", "method" : "gammaModeControl", "id" : 1}
<-- {"jsonrpc" : "2.0", "result" : {"gammaMode" : "ST.2084"}, "id" : 1}
--> {"jsonrpc" : "2.0", "method" : "gamma.ST2084Control", "id" : 1}
<-- {"jsonrpc" : "2.0", "result" : {"gamma.ST2084" : 0}, "id" : 1}

// to control gamma ST.2084
--> {"jsonrpc" : "2.0", "method" : "gamma.ST2084Control", "params" : {"gamma.ST2084" : 1}, "id" : 1}
<-- {"jsonrpc" : "2.0", "result" : {"gamma.ST2084" : 1}, "id" : 1}
```

3.1.6.28. Gamma HLG Control *[available with gammaMode : HLG]*

Method	Params(for data set)	Value
gamma.HLGControl	gamma.HLG	(-3) – 3

JSON-RPC call:

```
// --> : data sent to TV IPControl Server
// <-- : data sent to CI Client

// to get the gamma HLG info
--> {"jsonrpc" : "2.0", "method" : "gammaModeControl", "id" : 1}
<-- {"jsonrpc" : "2.0", "result" : {"gammaMode" : "HLG"}, "id" : 1}
--> {"jsonrpc" : "2.0", "method" : "gamma.HLGControl", "id" : 1}
<-- {"jsonrpc" : "2.0", "result" : {"gamma.HLG" : 0}, "id" : 1}

// to control gamma HLG
--> {"jsonrpc" : "2.0", "method" : "gamma.HLGControl", "params" : {"gamma.HLG" : 1}, "id" : 1}
<-- {"jsonrpc" : "2.0", "result" : {"gamma.HLG" : 1}, "id" : 1}
```

3.1.6.29. RGB Only Mode Control *[Deprecated from 2022TV]*

Method	Params(for data set)	Value
RGBOnlyModeControl	RGBOnlyMode	Off
RGBOnlyModeControl	RGBOnlyMode	Red
RGBOnlyModeControl	RGBOnlyMode	Green
RGBOnlyModeControl	RGBOnlyMode	Blue

JSON-RPC call:

```
// --> : data sent to TV IPControl Server
// <-- : data sent to CI Client

// to get the RGB only mode info
--> {"jsonrpc" : "2.0", "method" : "RGBOnlyModeControl", "id" : 1}
<-- {"jsonrpc" : "2.0", "result" : {"RGBOnlyMode" : "Off"}, "id" : 1}

// to control RGB only mode
--> {"jsonrpc" : "2.0", "method" : "RGBOnlyModeControl", "params" : {"RGBOnlyMode" : "Blue"}, "id" : 1}
<-- {"jsonrpc" : "2.0", "result" : {"RGBOnlyMode" : "Blue"}, "id" : 1}
```

3.1.6.30. Color Space Control

Method	Params(for data set)	Value
colorSpaceControl	colorSpace	Auto

Method	Params(for data set)	Value
colorSpaceControl	colorSpace	Native
colorSpaceControl	colorSpace	Custom

Note: In case of Custom mode, following sub options are available.

- Color
- Color Adjustment Point (only available on HDR mode)
- Red
- Green
- Blue

JSON-RPC call:

```
// --> : data sent to TV IPControl Server
// <-- : data sent to CI Client

// to get the color space info
--> {"jsonrpc" : "2.0", "method" : "colorSpaceControl", "id" : 1}
<-- {"jsonrpc" : "2.0", "result" : {"colorSpace" : "Native"}, "id" : 1}

// to control color space
--> {"jsonrpc" : "2.0", "method" : "colorSpaceControl", "params" : {"colorSpace" : "Custom"}, "id" : 1}
<-- {"jsonrpc" : "2.0", "result" : {"colorSpace" : "Custom"}, "id" : 1}
```

3.1.6.31. colorSpace.Color Control *[available with colorSpace : Custom]*

Method	Params(for data set)	Value
colorSpace.ColorControl	colorSpace.Color	Red
colorSpace.ColorControl	colorSpace.Color	Green
colorSpace.ColorControl	colorSpace.Color	Blue
colorSpace.ColorControl	colorSpace.Color	Yellow
colorSpace.ColorControl	colorSpace.Color	Cyan
colorSpace.ColorControl	colorSpace.Color	Magenta

JSON-RPC call:

```
// --> : data sent to TV IPControl Server
// <-- : data sent to CI Client

// to get the color space info, color info
--> {"jsonrpc" : "2.0", "method" : "colorSpaceControl", "id" : 1}
<-- {"jsonrpc" : "2.0", "result" : {"colorSpace" : "Custom"}, "id" : 1}
```

```
--> {"jsonrpc" : "2.0", "method" : "colorSpace.ColorControl", "id" : 1}
<-- {"jsonrpc" : "2.0", "result" : {"colorSpace.Color" : "Blue"}, "id" : 1}

// to control color info
--> {"jsonrpc" : "2.0", "method" : "colorSpace.ColorControl", "params" : {"colorSpace.Color" : "Red"},
"id" : 1}
<-- {"jsonrpc" : "2.0", "result" : {"colorSpace.Color" : "Red"}, "id" : 1}
```

3.1.6.32. colorSpace.ColorAdjustmentPoint Control *[available with colorSpace : Custom]*

Method	Params(for data set)	Value
colorSpace.ColorAdjustmentPointControl	colorSpace.ColorAdjustmentPoint	50%
colorSpace.ColorAdjustmentPointControl	colorSpace.ColorAdjustmentPoint	75%
colorSpace.ColorAdjustmentPointControl	colorSpace.ColorAdjustmentPoint	100%

- This methos is available only if colorSpace value is custom.
- This works only on CAL-NIGHT, CAL-DAY mode of pictureModeControl.

JSON-RPC call:

```
// --> : data sent to TV IPControl Server
// <-- : data sent to CI Client

// to get the color adjustment point value
--> {"jsonrpc" : "2.0", "method" : "colorSpace.ColorAdjustmentPointControl", "id" : 1}
<-- {"jsonrpc" : "2.0", "result" : {"colorSpace.ColorAdjustmentPoint" : "100%"}, "id" : 1}

// to control color adjustment point
--> {"jsonrpc" : "2.0", "method" : "colorSpace.ColorAdjustmentPointControl", "params" :
{"colorSpace.ColorAdjustmentPoint" : "50%"}, "id" : 1}
<-- {"jsonrpc" : "2.0", "result" : {"colorSpace.ColorAdjustmentPoint" : "50%"}, "id" : 1}
```

3.1.6.33. colorSpace.Red Control *[available with colorSpace : Custom]*

Method	Params(for data set)	Value
colorSpace.RedControl	colorSpace.Red	0 – 100

JSON-RPC call:

```
// --> : data sent to TV IPControl Server
// <-- : data sent to CI Client

// to get the color space info, red info
--> {"jsonrpc" : "2.0", "method" : "colorSpaceControl", "id" : 1}
<-- {"jsonrpc" : "2.0", "result" : {"colorSpace" : "Custom"}, "id" : 1}
--> {"jsonrpc" : "2.0", "method" : "colorSpace.RedControl", "id" : 1}
```

```

<-- {"jsonrpc" : "2.0", "result" : {"colorSpace.Red" : 0}, "id" : 1}

// to control red color
--> {"jsonrpc" : "2.0", "method" : "colorSpace.RedControl", "params" : {"colorSpace.Red" : 50}, "id" : 1}
<-- {"jsonrpc" : "2.0", "result" : {"colorSpace.Red" : 50}, "id" : 1}

```

3.1.6.34. colorSpace.Green Control *[available with colorSpace : Custom]*

Method	Params(for data set)	Value
colorSpace.GreenControl	colorSpace.Green	0 – 100

JSON-RPC call:

```

// --> : data sent to TV IPControl Server
// <-- : data sent to CI Client

// to get the color space info, green info
--> {"jsonrpc" : "2.0", "method" : "colorSpaceControl", "id" : 1}
<-- {"jsonrpc" : "2.0", "result" : {"colorSpace" : "Custom"}, "id" : 1}
--> {"jsonrpc" : "2.0", "method" : "colorSpace.GreenControl", "id" : 1}
<-- {"jsonrpc" : "2.0", "result" : {"colorSpace.Green" : 0}, "id" : 1}

// to control green color
--> {"jsonrpc" : "2.0", "method" : "colorSpace.GreenControl", "params" : {"colorSpace.Green" : 50}, "id" : 1}
<-- {"jsonrpc" : "2.0", "result" : {"colorSpace.Green" : 50}, "id" : 1}

```

3.1.6.35. colorSpace.Blue Control *[available with colorSpace : Custom]*

Method	Params(for data set)	Value
colorSpace.BlueControl	colorSpace.Blue	0 – 100

JSON-RPC call:

```

// --> : data sent to TV IPControl Server
// <-- : data sent to CI Client

// to get the color space info, blue info
--> {"jsonrpc" : "2.0", "method" : "colorSpaceControl", "id" : 1}
<-- {"jsonrpc" : "2.0", "result" : {"colorSpace" : "Custom"}, "id" : 1}
--> {"jsonrpc" : "2.0", "method" : "colorSpace.BlueControl", "id" : 1}
<-- {"jsonrpc" : "2.0", "result" : {"colorSpace.Blue" : 0}, "id" : 1}

// to control blue color
--> {"jsonrpc" : "2.0", "method" : "colorSpace.BlueControl", "params" : {"colorSpace.Blue" : 50}, "id" : 1}

```

```
: 1}
<-- {"jsonrpc" : "2.0", "result" : {"colorSpace.Blue" : 50}, "id" : 1}
```

3.1.7. Sound Control

Function: controls TV sound feature.

3.1.7.1. Sound Mode Control

Method	Params(for data set)	Value
soundModeControl	soundMode	Standard
soundModeControl	soundMode	Amplify
soundModeControl	soundMode	*Optimized <i>[Deprecated]</i>

Note: In case of using **AudioOutput/Optical, External** speaker, soundModeControl method does not work.

- "Optimized" option IS NOT available on 2019 Q-Series TV.
- "Optimized" option IS NOT available from 2020 TV.

JSON-RPC call:

```
// --> : data sent to TV IPControl Server
// <-- : data sent to CI Client

// to get the sound mode value
--> {"jsonrpc" : "2.0", "method" : "soundModeControl", "id" : 1}
<-- {"jsonrpc" : "2.0", "result" : {"soundMode" : "Standard"}, "id" : 1}

// to control sound mode
--> {"jsonrpc" : "2.0", "method" : "soundModeControl", "params" : {"soundMode" : "Amplify"}, "id" : 1}
<-- {"jsonrpc" : "2.0", "result" : {"soundMode" : "Amplify"}, "id" : 1}
```

3.1.7.2. Speaker Select Control

Method	Params(for data set)	Value
speakerSelectControl	speakerSelect	Internal
speakerSelectControl	speakerSelect	External*
speakerSelectControl	speakerSelect	AudioOut/Optical

Note : for external speaker control, use **"externalSpeakerControl"** method. This method only notifies the current speaker selection for external speaker case(set operation does not work).

JSON-RPC call:

```
// --> : data sent to TV IPControl Server
// <-- : data sent to CI Client

// to get the speaker select value
--> {"jsonrpc" : "2.0", "method" : "speakerSelectControl", "id" : 1}
<-- {"jsonrpc" : "2.0", "result" : {"speakerSelect" : "Internal"}, "id" : 1}

// to control speaker select
--> {"jsonrpc" : "2.0", "method" : "speakerSelectControl", "params" : {"speakerSelect" : "Internal"},
"id" : 1}
<-- {"jsonrpc" : "2.0", "result" : {"speakerSelect" : "Internal"}, "id" : 1}
```

3.1.7.3. External Speaker Control

Method	Params(for data set)	Value
externalSpeakerControl	deviceName	string
externalSpeakerControl	deviceId	integer

Note: externalSpeakerControl method DOES NOT search new device but uses the list already searched by the TV system because device search operation includes user interaction with huge delay for device pairing and connection.

TV System CAN NOT guarantee all kind of wireless speaker list due to the nature of the wireless connection characteristic.

JSON-RPC call:

```
// --> : data sent to TV IPControl Server
// <-- : data sent to CI Client

// to get the external speaker information
--> {"jsonrpc" : "2.0", "method" : "externalSpeakerControl", "id" : 1}
<-- {"jsonrpc" : "2.0", "result" : [{"deviceName" : "Samsung", "deviceId" : 1},
{"deviceName" : "Harman", "deviceId" : 2},
{"deviceName" : "JBL", "deviceId" : 3},
{"deviceName" : "Samsung", "deviceId" : 4}], "id" : 1}

// to control external speaker
--> {"jsonrpc" : "2.0", "method" : "externalSpeakerControl", "params" : {"deviceName" : "Samsung",
"deviceId" : 4}, "id" : 1}
<-- {"jsonrpc" : "2.0", "result" : {"deviceName" : "Samsung",
"deviceId" : 4}, "id" : 1}
```

3.1.8. Status

Function: gets system status.

3.1.8.1. Get TV States

Method [Get Only]	Params(for data set)
getTVStates	N/A

Note: If some of TV states information reading is failed due to system policy, that field may be omitted.

Params(for data set)	Value
speakerSelect	Internal
speakerSelect	External*
speakerSelect	AudioOut/Optical*

Note: For AudioOut/Optical , External speaker usage case, TV may not provide all sound related informations.

Params(for data get)	Value
volume	0 - 100

Params(for data get)	Value
mute	muteOff
mute	muteOn

Params(for data set)	Value
inputSource	TV
inputSource	HDMI1
inputSource	HDMI2
inputSource	HDMI3
inputSource	HDMI4
inputSource	AV1
inputSource	COMPONENT1
inputSource	USB*
inputSource	RVU* <i>[Deprecated]</i>

Note: for **USB/RVU** source control, use **USBSourceControl/RVUSourceControl**. "inputSourceControl" method only notifies the current source name for USB/RVU source case(set operation will not work since there is USB/RVU device list).

Params(for data set)	Value
pictureSize	16:9
pictureSize	4:3

Params(for data set)	Value
pictureMode	Dynamic


```
"pictureSize" : "16:9",  
"pictureMode" : "Dynamic"}, "id" : 1}
```

3.1.8.2. Get Video States

Method [Get Only]	Params(for data set)
getVideoStates	N/A

Note: If some of all video related states information reading is failed due to system policy, that field may be omitted.

Params(for data get)	Value
contrast	0 - 50

Params(for data get)	Value
brightness	(-5) – 5

Note: brightness range is -5 to 5.

Params(for data get)	Value
sharpness	0 - 20

Params(for data get)	Value
color	0 - 50

Params(for data get)	Value
tint	(-15) - 15

JSON-RPC call:

```
// --> : data sent to TV IPControl Server  
// <-- : data sent to CI Client  
  
// to get the Video States  
--> {"jsonrpc" : "2.0", "method" : "getVideoStates", "id" : 1}  
<-- {"jsonrpc" : "2.0", "result" : {"contrast" : 50,  
    "brightness" : 3,  
    "sharpness" : 25,  
    "color" : 25,  
    "tint" : 0}, "id" : 1}
```

3.1.9. Remote Key Control

Function: sends virtual remote key to the TV.

Method [Set Only]	Params(for data set)	Value
remoteKeyControl	remoteKey	power
remoteKeyControl	remoteKey	cursorUp
remoteKeyControl	remoteKey	cursorDn
remoteKeyControl	remoteKey	cursorLeft
remoteKeyControl	remoteKey	cursorRight
remoteKeyControl	remoteKey	menu
remoteKeyControl	remoteKey	firstScreen
remoteKeyControl	remoteKey	enter
remoteKeyControl	remoteKey	fastforward
remoteKeyControl	remoteKey	rewind
remoteKeyControl	remoteKey	play
remoteKeyControl	remoteKey	stop
remoteKeyControl	remoteKey	pause
remoteKeyControl	remoteKey	return
remoteKeyControl	remoteKey	exit
remoteKeyControl	remoteKey	number1
remoteKeyControl	remoteKey	number2
remoteKeyControl	remoteKey	number3
remoteKeyControl	remoteKey	number4
remoteKeyControl	remoteKey	number5
remoteKeyControl	remoteKey	number6
remoteKeyControl	remoteKey	number7
remoteKeyControl	remoteKey	number8
remoteKeyControl	remoteKey	number9
remoteKeyControl	remoteKey	number0
remoteKeyControl	remoteKey	caption
remoteKeyControl	remoteKey	dash
remoteKeyControl	remoteKey	red
remoteKeyControl	remoteKey	green
remoteKeyControl	remoteKey	yellow

Method [Set Only]	Params(for data set)	Value
remoteKeyControl	remoteKey	blue
remoteKeyControl	remoteKey	ambient
remoteKeyControl	remoteKey	multiview

Note: If ambient mode is ON, only "firstScreen" key value works for ambient mode exit. "exit" key is not effective for ambient mode exit. This is same behavior with Samsung TV Smart remote controller.

Note: multiview key is available only if product type is MICRO LED.

Note: If it is not a supported remoteKey, it may not work regardless of whether the key input is successful.

JSON-RPC call:

```
// --> : data sent to TV IPControl Server
// <-- : data sent to CI Client

// to set a virtual remote key control
--> {"jsonrpc" : "2.0", "method" : "remoteKeyControl", "params" : {"remoteKey" : "menu"}, "id" : 1}
<-- {"jsonrpc" : "2.0", "result" : {"remoteKey" : "menu"}, "id" : 1}
```

3.1.10. App Control

Function: directly launches TV application.

Method	Params(for data set)	Value
directAccessControl	applicationName	webBrowser
directAccessControl	applicationName	netflix
directAccessControl	applicationName	amazon
directAccessControl	applicationName	pandora
directAccessControl	applicationName	vudu
directAccessControl	applicationName	youTube
directAccessControl	applicationName	hulu
directAccessControl	url*	(string)

Note: TV may not support directAccessControl for all applications since there maybe not-installed application: pandora, vudu, hulu are not pre-installed applications. **"url" param could be omitted in case of no need.**

For get operation, TV only tells the applicationName which is listed in this specification. If current running application name is not listed in here, TV tells "unknown".

If the TV system is switching application mode, client may have "unknown" states.

JSON-RPC call:

```
// --> : data sent to TV IPControl Server
// <-- : data sent to CI Client

// to get the current TV application name
--> {"jsonrpc" : "2.0", "method" : "directAccessControl", "id" : 1}
<-- {"jsonrpc" : "2.0", "result" : {"applicationName" : "youTube"}, "id" : 1}

// to launch TV application directly
--> {"jsonrpc" : "2.0", "method" : "directAccessControl", "params" : {"applicationName" :
"webBrowser",
"url" : "www.google.com"}, "id" : 1}
<-- {"jsonrpc" : "2.0", "result" : {"applicationName" : "webBrowser",
"url" : "www.google.com"}, "id" : 1}
```

3.1.11. First Screen App Control

Function: directly launches TV application located in the TV's first screen.

Method	Params(for data set)	Value
firstScreenAppControl	applicationName	

Note: Client device must get application list first to know what kind of apps are available.

TV provides app list of TV first screen which can be modified by user demand.

Client should select only one of the first screen application list for app launch.

If TV's app list includes Unicode character, Content-length of HTTP header should be counted based on the UTF-8 encoding.

This is supported from 2021 product.

JSON-RPC call:

```
// --> : data sent to TV IPControl Server
// <-- : data sent to CI Client

// to get the TV first screen application list
--> {"jsonrpc" : "2.0", "method" : "firstScreenAppControl", "id" : 1}
<-- {"jsonrpc" : "2.0", "result" : [{"applicationName" : "YouTube"},
{"applicationName" : "Netflix"}], "id" : 1}

// to launch TV application directly
--> {"jsonrpc" : "2.0", "method" : "firstScreenAppControl", "params" : {"applicationName" :
"Netflix"}, "id" : 1}
<-- {"jsonrpc" : "2.0", "result" : {"applicationName" : "Netflix"}, "id" : 1}
```

3.1.12. Art Mode Control

Function: controls Art Mode for Frame TV only.

In this mode, TV settings change is not allowed. To change TV settings, user needs to exit Art Mode.

Method	Params(for data set)	Value
artModeControl	artMode	artModeOff
		artModeOn

Note: Only Frame TV can support artModeControl. If it's not, client will get error message.

JSON-RPC call:

```
// --> : data sent to TV IPControl Server
// <-- : data sent to CI Client

// to get the current Art Mode state
--> {"jsonrpc" : "2.0", "method" : "artModeControl", "id" : 1}
<-- {"jsonrpc" : "2.0", "result" : {"artMode" : "artModeOn"}, "id" : 1}

// to handle Art Mode
--> {"jsonrpc" : "2.0", "method" : "artModeControl", "params" : {"artMode" : "artModeOn"}, "id" : 1}
<-- {"jsonrpc" : "2.0", "result" : {"artMode" : "artModeOn"}, "id" : 1}
```

3.1.13. MultiView Control

Function: client can control TV multi-view modes.

Method	Params(for data set)	Value
multiviewControl	multiviewMode	

Note: Client device must get multi-view modes first to see TV view modes.

Client should select only one of the multi-view modes for view mode control.

This is supported from 2021 product.

JSON-RPC call:

```
// --> : data sent to TV IPControl Server
// <-- : data sent to CI Client

// to get the TV MultiView modes first
--> {"jsonrpc" : "2.0", "method" : "multiviewControl", "id" : 1}
<-- {"jsonrpc" : "2.0", "result" : [{"multiviewMode" : "Saved0"},
{"multiviewMode" : "Saved1"},
{"multiviewMode" : "Saved2"},
{"multiviewMode" : "Saved3"}], "id" : 1}

// to control TV view mode
--> {"jsonrpc" : "2.0", "method" : "multiviewControl", "params" : {"multiviewMode" : "Saved1"}, "id" : 1}
<-- {"jsonrpc" : "2.0", "result" : {"multiviewMode" : "Saved1"}, "id" : 1}
```

3.1.14. Display Rotator Control

Function: client can control TV display rotator

Method	Params(for data set)	Value
displayRotatorControl	orientation	Portrait
displayRotatorControl	orientation	Landscape

Note: This is supported from 2022 products. TV should be configured with auto rotation accessory or it should be Sero.

Note: IPControl client can get the exact state of the display rotator only if the requested rotation operation is completed because TV needs to take enough time to complete the operation.

JSON-RPC call:

```
// --> : data sent to TV IPControl Server
// <-- : data sent to CI Client

// to get the Display Rotator status
--> {"jsonrpc" : "2.0", "method" : "displayRotatorControl", "id" : 1}
<-- {"jsonrpc" : "2.0", "result" : {"orientation" : "Portrait"}, "id" : 1}

// to control Display Rotator
--> {"jsonrpc" : "2.0", "method" : "displayRotatorControl", "params" : {"orientation" : "Landscape"},
    "id" : 1}
<-- {"jsonrpc" : "2.0", "result" : {"orientation" : "Landscape"}, "id" : 1}
```

3.1.15. Hotel TV Control

Function: client can control various options of the Hotel TV.

Note: These methods are supported on the Hotel TV only.

Note: To use Soft AP related features, Hotel TV should be connected via wired network because Soft AP uses wireless network interface. Hotel TV WiFi driver does not support host AP and wireless network connection simultaneously.

Control	Method#	Method Name
Hotel TV	56	getHTVInformation
Hotel TV	57	getHTVNetworkInformation
Hotel TV	58	duplicateHTVConfigControl
Hotel TV	59	HTVFactoryLockControl
Hotel TV	60	HTVRoomStatusControl
Hotel TV	61	setHTVTime

Control	Method#	Method Name
Hotel TV	62	showHTVNotification
Hotel TV	63	softAPStatusControl
Hotel TV	64	softAPSSIDControl
Hotel TV	65	getSoftAPSecurityKey
Hotel TV	67	softAPWiFiChannelControl
Hotel TV	68	softAPSignalLevelControl
Hotel TV	69	forwardMessage

3.1.15.1. Get HTV Information

Function: client can get HTV information (room number, FW version, clone version, HTV mode, model name, submicom version).

Method [Get Only]	Params(for data set)
getHTVInformation	N/A
Params(for data get)	Value
roomNumber	String
Params(for data get)	Value
firmwareVersion	String
Params(for data get)	Value
cloneVersion	String (Format : "#.#")

Note: cloneVersion value type is String. But, it should be double type format ("1.0", "1.1", ...) because HTV stores the version as double inside of the system.

Params(for data get)	Value
htvMode	IPTV
htvMode	RFTV

Note: If Hbrowser mode is ON, then its IPTV, else RFTV.

Params(for data get)	Value
modelName	String
Params(for data get)	Value
subMicomVersion	String

JSON-RPC call:


```
// --> : data sent to TV IPControl Server
// <-- : data sent to CI Client

// to get the HTV Information
--> {"jsonrpc" : "2.0", "method" : "getHTVInformation", "id" : 1}
<-- {"jsonrpc" : "2.0", "result" : {"roomNumber" : "101",
"firmwareVersion" : "XXX",
"cloneVersion" : "1.0",
"htvMode" : "IPTV",
"modelName" : "HF690",
"subMicomVersion" : "XXX"}, "id" : 1}
```

3.1.15.2 Get HTV Network Information

Function: client can get HTV network information (MAC address, IP address, IP mode).

Method [Get Only]	Params(for data set)
getHTVNetworkInformation	N/A

Params(for data get)	Value
macAddress	String (Format : "##:##:##:##:##:##")

Params(for data get)	Value
ipAddress	String (Format : "###.###.###.###")

Params(for data get)	Value
ipMode	DHCP
ipMode	STATIC

JSON-RPC call:

```
// --> : data sent to TV IPControl Server
// <-- : data sent to CI Client

// to get the HTV Network Information
--> {"jsonrpc" : "2.0", "method" : "getHTVNetworkInformation", "id" : 1}
<-- {"jsonrpc" : "2.0", "result" :
      {"macAddress" : "12:34:56:78:90:AB",
       "ipAddress" : "192.168.0.2",
       "ipMode" : "DHCP"}, "id" : 1}
```

3.1.15.3. Duplicate HTV Config Control

Function: client can send URL for duplicating HTV configuration

Method	Params(for data set)	Value
duplicateHTVConfigControl	cloneUrl	String
duplicateHTVConfigControl	cloneVersion	String (Format : "#.#")
duplicateHTVConfigControl	versionCheck	String (yes or no)

Note: cloneVersion value type is String. But, it should be double type format ("1.0", "1.1", ...) because HTV stores the version as double inside of the system. This information will be used in the cloneVersion param of getHTVInformation method.

Note: Valid URL and clone file should be provided by client. After clone completion, HTV should be rebooted. Otherwise, some settings may not be reflected(e.g : some factory settings etc.).

Note: The cloneUrl information is used for IPControl server to get the configure file from client. Client should provide HTTP or HTTPS server for configuration file sharing. IPControl server will pass cloneUrl information to HTV system so that HTV system will download the configuration file from the HTTP/HTTPS server. Once duplicateHTVConfigControl method is set to IPControl server side, client can check current clone version using "cloneVersion of the getHTVInformation response".

Note: use the following form for clone url.

```
"cloneUrl" : "http://###. ###. ###. ###/clone/clone_url.txt"
```

the clone_url.txt and .zip files must be under the clone directory.

the content of clone_url.txt is configured as follows.

- The .zip name of the 'Clone' file obtained from TV after create the Clone image.
- Specify the .zip file size.

For Example, "T-RSLFAKUC-0090. zip=602957" in clone_url.txt

JSON-RPC call:

```
// --> : data sent to TV IPControl Server
// <-- : data sent to CI Client

// to set the clone URL of the client ( 172.18.103.109 is just an example )
--> {"jsonrpc" : "2.0", "method" : "duplicateHTVConfigControl",
"params" : {"cloneUrl" : "https://172.18.103.109/clone/clone_url.txt",
"cloneVersion" : "1.0", "versionCheck" : "yes"}, "id" : 1}
<-- {"jsonrpc" : "2.0", "result" :
{"cloneUrl" : "https://172.18.103.109/clone/clone_url.txt" ,
"cloneVersion" : "1.0", "versionCheck" : "yes"}, "id" : 1}
```

Note: This command is logic-applied with a timeout of 60 seconds. Server error is returned at timeout, and for other errors, the following error is returned.

ErrorCode	ErrorMsg
1001	Firmware Version Error

ErrorCode	ErrorMsg
1002	Model Name Error
1003	Firmware Code Error
1003	Execution Error

```
// --> : data sent to TV IPControl Server
// <-- : data sent to CI Client

// to set the clone URL of the client
--> {"jsonrpc" : "2.0", "method" : "duplicateHTVConfigControl",
"params" : {"cloneUrl" : "https://###.###.###.###/cloneFile",
            "cloneVersion" : "1.0"}, "id" : 1}
<-- {"jsonrpc" : "2.0", "result" :
      {"ErrorCode" : 1001, "ErrorMsg" : "Firmware Version Error"}, "id" : 1}
```

3.1.15.4. HTV Factory Lock Control

Function: client can enable/disable HTV Factory Lock option.

Method	Params(for data set)	Value
HTVFactoryLockControl	HTVFactoryLock	On
HTVFactoryLockControl	HTVFactoryLock	Off

JSON-RPC call:

```
// --> : data sent to TV IPControl Server
// <-- : data sent to CI Client

// to get the HTV Factory Lock status
--> {"jsonrpc" : "2.0", "method" : "HTVFactoryLockControl", "id" : 1}
<-- {"jsonrpc" : "2.0", "result" : {"HTVFactoryLock" : "On"}, "id" : 1}

// to control HTV Factory Lock
--> {"jsonrpc" : "2.0", "method" : "HTVFactoryLockControl", "params" : {"HTVFactoryLock" : "Off"},
"id" : 1}
<-- {"jsonrpc" : "2.0", "result" : {"HTVFactoryLock" : "Off"}, "id" : 1}
```

3.1.15.5. HTV Room Status Control

Function: client can delete room guest accounts and credentials after setting this option as CheckedOut.

Method	Params(for data set)	Value
HTVRoomStatusControl	roomStatus	CheckedIn

Method	Params(for data set)	Value
HTVRoomStatusControl	roomStatus	CheckedOut

JSON-RPC call:

```
// --> : data sent to TV IPControl Server
// <-- : data sent to CI Client

// to get the HTV Room status
--> {"jsonrpc" : "2.0", "method" : "HTVRoomStatusControl", "id" : 1}
<-- {"jsonrpc" : "2.0", "result" : {"roomStatus" : "CheckedIn"}, "id" : 1}

// to control HTV Room status
--> {"jsonrpc" : "2.0", "method" : "HTVRoomStatusControl", "params" : {"roomStatus" : "CheckedOut"},
"id" : 1}
<-- {"jsonrpc" : "2.0", "result" : {"roomStatus" : "CheckedOut"}, "id" : 1}
```

3.1.15.6. Set HTV Time

Function: client can setup HTV time by using yyyyymmddhhmmss format.

Method	Params(for data set)	Value
setHTVTime	time	String

Note: time value should be yyyyymmddhhmmss format.

20230615120000 value means 2023/06/15 12:00:00.

params	Value Range
Year (yyyy)	2001 <= yyyy <= 2099
Month (mm)	1 <= mm <= 12
Day (dd)	1 <= dd <= LastDay of that month (leap year Feb. is 29, otherwise is 28, Apr. & Jun. & Sep. & Nov. are 30, other months are 31)
Hour (hh)	0 <= hh <= 23
Minute (mm)	0 <= mm <= 59
Second (ss)	0 <= ss <= 59

JSON-RPC call:

```
// --> : data sent to TV IPControl Server
// <-- : data sent to CI Client
```

```
// to control HTV Time
--> {"jsonrpc" : "2.0", "method" : "setHTVTime", "params" : {"time" : "20230615120000"}, "id" : 1}
<-- {"jsonrpc" : "2.0", "result" : {"time" : "20230615120000"}, "id" : 1}
```

3.1.15.7. Show HTV Notification

Function: client can display a notification on the HTV.

Method	Params(for data set)	Value
showHTVNotification	message	String

Note: HTV should be ON. The message information will be shared for HTV notification displaying.

Note: Client can display up to 200 characters.

JSON-RPC call:

```
// --> : data sent to TV IPControl Server
// <-- : data sent to CI Client

// to show a notification
--> {"jsonrpc" : "2.0", "method" : "showHTVNotification", "params" : {
"message" : "Welcome to our hotel."}, "id" : 1}
<-- {"jsonrpc" : "2.0", "result" : {"message" : "Welcome to our hotel."}, "id" : 1}
```

3.1.15.8. SoftAP Status Control

Function: client can enable/disable SoftAP of the HTV.

Note : HTV Wired network connection is needed.

Note : softapStatus option can not be turned on if HTV is on wireless network.

Method	Params(for data set)	Value
softAPStatusControl	softapStatus	On
softAPStatusControl	softapStatus	Off

JSON-RPC call:

```
// --> : data sent to TV IPControl Server
// <-- : data sent to CI Client

// to get the SoftAP status
--> {"jsonrpc" : "2.0", "method" : "softAPStatusControl", "id" : 1}
<-- {"jsonrpc" : "2.0", "result" : {"softapStatus" : "On"}, "id" : 1}

// to control SoftAP status
--> {"jsonrpc" : "2.0", "method" : "softAPStatusControl", "params" : {"softapStatus" : "Off"}, "id" : 1}
```

```
1}
<-- {"jsonrpc" : "2.0", "result" : {"softapStatus" : "Off"}, "id" : 1}
```

3.1.15.9. SoftAP SSID Control

Function: Client can control SSID of SoftAP.

Method	Params(for data set)	Value
softAPSSIDControl	softapSsid	String

JSON-RPC call:

```
// --> : data sent to TV IPControl Server
// <-- : data sent to CI Client

// to get the SoftAP SSID status
--> {"jsonrpc" : "2.0", "method" : "softAPSSIDControl", "id" : 1}
<-- {"jsonrpc" : "2.0", "result" : {"softapSsid" : "HotelTV123"}, "id" : 1}

// to control SoftAP SSID
--> {"jsonrpc" : "2.0", "method" : "softAPSSIDControl", "params" : {"softapSsid" : "HotelTV456"}, "id" : 1}
<-- {"jsonrpc" : "2.0", "result" : {"softapSsid" : "HotelTV456"}, "id" : 1}
```

3.1.15.10. Get SoftAP Security Key

Function: Client can get security key of SoftAP which is generated previously.

Method [Get Only]	Params(for data set)
getSoftAPSecurityKey	N/A
Params(for data get)	Value
sofatpSecurityKey	String

JSON-RPC call:

```
// --> : data sent to TV IPControl Server
// <-- : data sent to CI Client

// to get Soft AP Security Key
--> {"jsonrpc" : "2.0", "method" : "getSoftAPSecurityKey", "id" : 1}
<-- {"jsonrpc" : "2.0", "result" : {"sofatpSecurityKey" : "123456"}, "id" : 1}
```

3.1.15.11. SoftAP WiFi Channel Control

Function: Client can control WiFi channel of SoftAP.

Method	Params(for data set)	Value
softAPWiFiChannelControl	softapChannel	String

JSON-RPC call:

```
// --> : data sent to TV IPControl Server
// <-- : data sent to CI Client

// to get the SoftAP WiFi Channel status
--> {"jsonrpc" : "2.0", "method" : "softAPWiFiChannelControl", "id" : 1}
<-- {"jsonrpc" : "2.0", "result" : {"softapChannel" : "0"}, "id" : 1}

// to control SoftAP WiFi Channel
--> {"jsonrpc" : "2.0", "method" : "softAPWiFiChannelControl", "params" : {"softapChannel" : "1"},
"id" : 1}
<-- {"jsonrpc" : "2.0", "result" : {"softapChannel" : "1"}, "id" : 1}
```

3.1.15.12. SoftAP Signal Level Control

Function: Client can control signal level of SoftAP.

Note : HTV Wired network connection is needed.

Note : SoftAP should be enabled on the HTV.

Method	Params(for data set)	Value
softAPSignalLevelControl	softapSignalLevel	String

JSON-RPC call:

```
// --> : data sent to TV IPControl Server
// <-- : data sent to CI Client

// to get the SoftAP Signal Level status
--> {"jsonrpc" : "2.0", "method" : "softAPSignalLevelControl", "id" : 1}
<-- {"jsonrpc" : "2.0", "result" : {"softapSignalLevel" : "50"}, "id" : 1}

// to control SoftAP Signal Level
--> {"jsonrpc" : "2.0", "method" : "softAPSignalLevelControl", "params" : {"softapSignalLevel" :
"10"}, "id" : 1}
<-- {"jsonrpc" : "2.0", "result" : {"softapSignalLevel" : "10"}, "id" : 1}
```

3.1.15.13. Update Firmware

Function: Updates the device firmware.

Method	Params(for data set)	Value
updateFirmware	fileName	String

Method	Params(for data set)	Value
updateFirmware	SWVersion	String
updateFirmware	URL	String
updateFirmware	size	String

JSON-RPC call:

```
// --> : data sent to TV IPControl Server

// to process updating firmware
--> {"jsonrpc" : "2.0", "method" : "updateFirmware",
    "params" : {"fileName" : "swuimage.bem",
    "SWVersion" : "011",
    "URL" : "http://107.108.175.192:8080/data/T-MST10PDEUCB_SECUSB/image/swuimage.bem",
    "size" : "1246792013"}, "id" : 1}
<-- {"jsonrpc" : "2.0", "result" :
    {"fileName" : "swuimage.bem",
    "SWVersion" : "011",
    "URL" : "http://107.108.175.192:8080/data/T-MST10PDEUCB_SECUSB/image/swuimage.bem",
    "size" : "1246792013"}, "id" : 1}
```

3.1.15.14. Forward Message

Function: Forwarding message to TEP application

Method	Params(for data set)	Value
forwardMessage	message	String

Note: HTV should be ON.

Note: Client can display up to 200 characters.

JSON-RPC call:

```
// --> : data sent to TV IPControl Server
// <-- : data sent to CI Client

// to forward message
--> {"jsonrpc" : "2.0", "method" : "forwardMessage", "params" : {"message" : "Hello"}, "id" : 1}
<-- {"jsonrpc" : "2.0", "result" : {"message" : "Hello"}, "id" : 1}
```


4. Revision History

Version	Contents	Date	Author
0.7	TV IP Control Protocol Draft Ver.0.7	2016.11.25	Shawn SH Park
0.8	Renamed 'Command' as 'Control', 'Method'. Updated Samsung IP Control server error code. Added WoL/WoW for powerControl. Added USBSourceControl. Added RVUSourceControl. Added externalSpeakerControl. Updated directAccessControl for get operation.	2017.01.02	Shawn SH Park
0.8a	Updated Samsung IP Control server error code description.	2017.01.03	Shawn SH Park
0.8b	Added "SmartAudio" in the soundModeControl.	2017.01.04	Shawn SH Park
0.8c	Updated USB deviceId data type as string.		
Updated RVU deviceId data type as string.	2017.01.13	Shawn SH Park	
0.9	Added number keys in the remoteKeyControl. Added new input sources in the inputSourceControl. Updated "SmartAudio" as "Optimized" in the soundModeControl.	2017.02.07	Shawn SH Park
0.9a	Added a description in the directChannelControl.	2017.03.13	Shawn SH Park
0.9b	Updated getTVStates for "AudioOut/Optical", "External" speaker modes. Added a description in the directVolumeControl. Added a description in the muteControl. Added a description in the soundMode. Updated description of USBSourceControl. Added a description in the channelUpDnControl.	2017.03.16	Shawn SH Park
0.9c	Added "power" key in the remoteKeyControl	2017.03.17	Shawn SH Park
0.9d	Updated brightness value range to "(-5) ~ 5" for brightnessControl, getVideoStates.	2017.03.20	Shawn SH Park
0.9e	Added cipher suites information for preventing weak and vulnerable security.	2017.03.21	Shawn SH Park
1.0	Added limitation for getTVStates, getVideoStates(may omit some of the informations depending on the availability).	2017.04.05	Shawn SH Park

Version	Contents	Date	Author
	Added caption, dash, red, green, yellow, blue key in the remoteKeyControl. Added "powerOn" option in the powerControl. Updated control list table.		
1.1	Added artModeControl for frame TV support.	2017.05.30	Shawn SH Park
1.2	[Spec. is updated for 2018 TVs] Updated value for contrastControl / sharpnessControl / colorControl / tintControl. Added "ambient" key in the remoteKeyControl. RVUSourceControl is deprecated from version 1.2 since 2018 TV does not have RVU feature.	2018.01.09	Shawn SH Park
1.3	[Spec. is updated for 2019 TVs] Removed HDR+ option in 3.1.6.1 / 3.1.8.1 since 2019 TV does not support HDR+ option. Added sound mode option limitation for 2019 Q-series in soundModeControl / getTVStates (No "Optimized" Option for QTV). Added note for handling ambient mode in remoteKeyControl	2018.12.19	Shawn SH Park
1.4	Added CAL-NIGHT, CAL-DAY option in the pictureModeControl. Added below methods in pictureControl : 3.1.6.8 pictureCalibrationModeControl 3.1.6.9 backlightControl 3.1.6.10 digitalCleanViewControl 3.1.6.11 autoMotionPlusControl 3.1.6.12 AMP.BlurReductionControl 3.1.6.13 AMP.JudderReductionControl 3.1.6.14 AMP.LEDClearMotionControl 3.1.6.15 localDimmingControl 3.1.6.16 filmModeControl 3.1.6.17 contrastEnhancerControl 3.1.6.18 colorToneControl 3.1.6.19 WB2PointControl 3.1.6.20 WB20PointControl 3.1.6.21 WB20P.IntervalControl 3.1.6.22 WB20P.RedControl 3.1.6.23 WB20P.GreenControl 3.1.6.24 WB20P.BlueControl 3.1.6.25 gammaModeControl 3.1.6.26 gamma.BT1886Control 3.1.6.27 gamma.ST2084Control 3.1.6.28 gamma.HLGControl 3.1.6.29 RGBOnlyModeControl 3.1.6.30 colorSpaceControl 3.1.6.31 colorSpace.ColorControl 3.1.6.32 colorSpace.ColorAdjustmentPointControl	2019.04.04	Shawn SH Park

Version	Contents	Date	Author
	3.1.6.33 colorSpace.RedControl 3.1.6 34 colorSpace.GreenControl 3.1.6.35 colorSpace.BlueControl		
1.5	Fixed typo error on WB20PointModeControl, WB20P.IntervalControl, WB20P.RedControl, WB20P.GreenControl, WB20P.BlueControl. Updated sound mode option limitation for 2020 TVs in soundModeControl / getTVStates (No "Optimized" Option). Marked gray highlight on deprecated features of TV. Added gray highlighted descriptions for the renamed options of 2020 TV menu. Added FilmmakerMode in pictureModeControl. Added Appendix and appendix 5.1 for 2020 TV menu tree changes. Added a support TV table for pictureControl. Added Appendix 5.2 for menu name change (Backlight -> Brightness, Brightness -> Shadow Detail)	2019.12.23	Shawn SH Park
1.6	Updated port number info.	2020.01.29	Shawn SH Park
1.7	Added a note for inputSourceControl.	2020.02.25	Shawn SH Park
1.8	Added 3.1.11 First Screen App Control. Updated inputSourceControl for HDBaseT on the Terrace series.	2020.10.20	Shawn SH Park
1.9	Added 3.1.13 MultiView Control. Added "Multiview" key at 3.1.9 Remote Key Control.	2020.11.12	Shawn SH Park
1.10	[Spec. is updated for 2022 TVs] Added 3.1.14 Display Rotator Control. Updated directChannelControl for TVPlus support.	2021.11.02	Shawn SH Park
1.11	Deprecated – RGBOnlyModeControl Deprecated item – Natural item in the pictureModeControl	2021.11.11	Shawn SH Park

Version	Contents	Date	Author
2.0	Update table of 3.1 for Hotel TV support. Added 3.1.15 Hotel TV Control : 3.1.15.1 Get HTV Information 3.1.15.2 Get HTV Network Information 3.1.15.3 Duplicate HTV Config Control 3.1.15.4 HTV Factory Lock Control 3.1.15.5 HTV Room Status Control 3.1.15.6 Set HTV Time 3.1.15.7 Show HTV Notification 3.1.15.8 SoftAP Status Control 3.1.15.9 SoftAP SSID Control 3.1.15.10 Get SoftAP Security Key 3.1.15.11 SoftAP WiFi Channel Control 3.1.15.12 SoftAP Signal Level Control	2023.08.02	Shawn SH Park
2.1	[Spec. is updated for 2024 TVs] Added note about LPM feature at 3.1.1 Power Control Change description about LPM feature at 3.1.1 Power Control Convert docx to md Change description about HDMI ports at 3.1.5.1. Input Source Control	2024.02.16	Seockhun Bae
2.2	Add iptv param at 3.1.4.1. DirectChannelControl Method Add updateFirmware Method as 3.1.15.13	2024.03.13	Seockhun Bae
2.3	Update 3.1.4.1. DirectChannelControl Method	2024.04.16	Seockhun Bae
2.4	Update 3.1.15.1. htvMode description	2024.06.26	Seockhun Bae
2.5	Update 3.1.15.3. Duplicate HTV Config Control - timeout added - errorType are added	2025.01.14	Seockhun Bae
2.6	Add command 3.1.15.4. Forward Message	2025.04.30	Seockhun Bae
2.7	Delete phrases related to LPM	2025.06.02	Seockhun Bae
2.8	Add note on remoteKeyControl, contrastControl	2025.11.20	Seockhun Bae

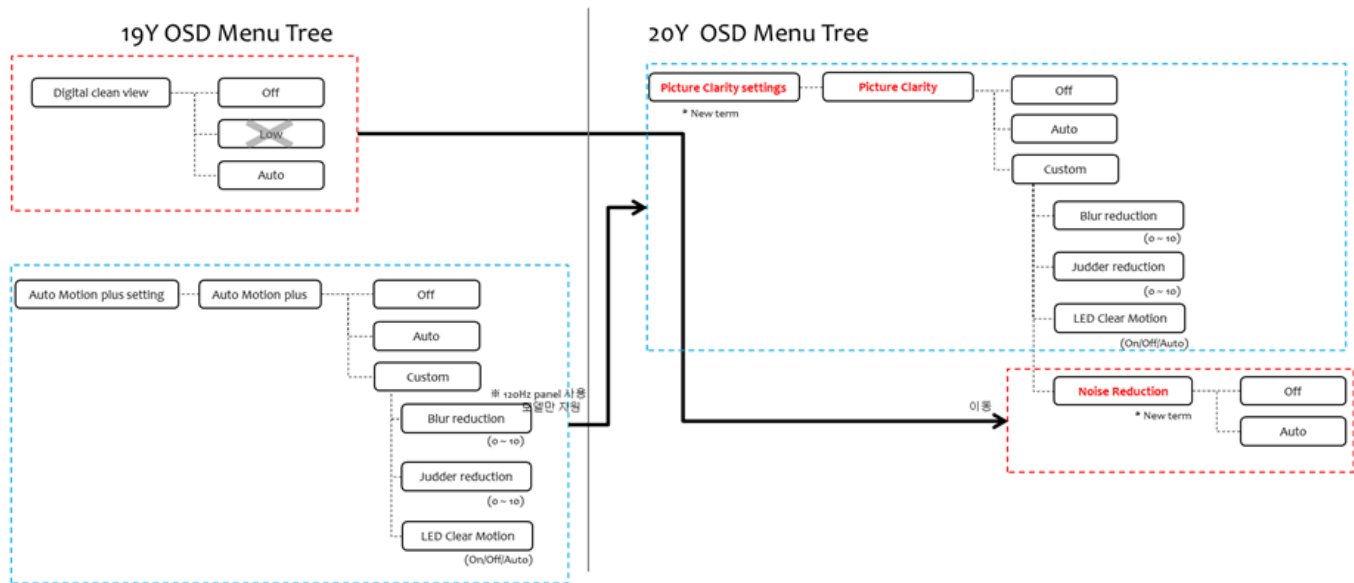
5. Appendix

5.1. TV menu tree change for 2020 TV

CONCEPT & KEY FEATURES

Case 1

With Auto motion plus menu

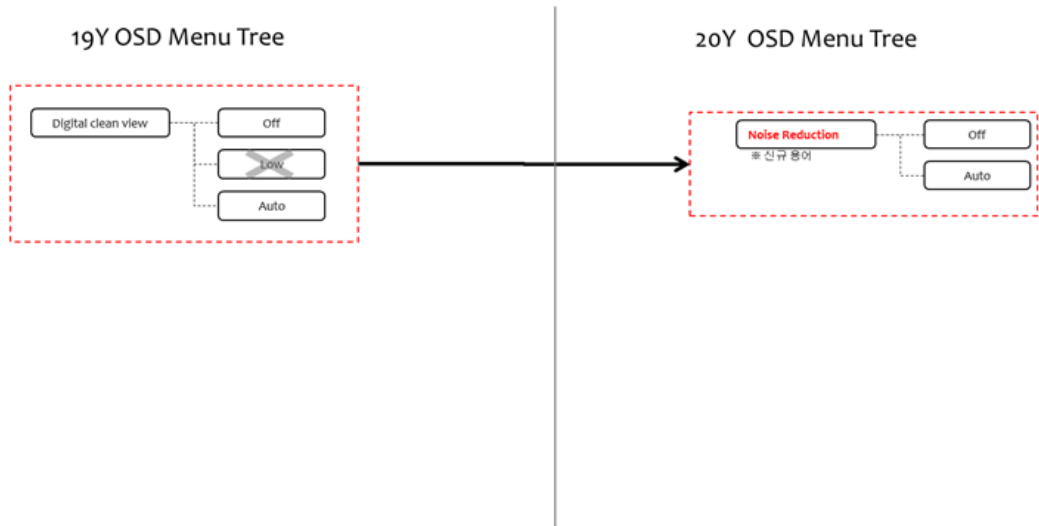


SAMSUNG I CONFIDENTIAL

CONCEPT & KEY FEATURES

Case 2

Without Auto motion plus menu



SAMSUNG I CONFIDENTIAL

5.2. TV menu name change for 2020 TV

